

PARA O ALTO E AVANTE

PERÍCIA FORENSE EM SISTEMAS IN VIVO II



4

LISTA DE FIGURAS

Figura 1 – O modelo EDRM	8
Figura 2 – Representação simplificada do processo pericial	13
Figura 3 – Topologia geral do cenário.....	14
Figura 4 – Verificação do servidor SSH	15
Figura 5 – Informações do ambiente	16
Figura 6 – Criação do vtype	19
Figura 7 – Arquivo module.dwarf.....	19
Figura 8 – Finalização do perfil	20
Figura 9 – Transferência do perfil.....	20
Figura 10 – Verificando os perfis disponíveis	21
Figura 11 – Módulo carregável pelo kernel	22
Figura 12 – Criação do primeiro socket.....	23
Figura 13 – <i>Hashes</i> do <i>dump</i>	24
Figura 14 – Listagem parcial de <i>plug-ins</i>	26
Figura 15 – Arquivo Bash2.txt	27
Figura 16 – Arquivo Psaux2.txt	27
Figura 17 – Arquivo Id_User.txt.....	28
Figura 18 – Arquivo Psaux3.txt	28
Figura 19 – Arquivo Netstat2.txt	29
Figura 20 – Timeline das ações	31
Figura 21 – Posicionamento para captura.....	34
Figura 22 – Saída do comando ifconfig	37
Figura 23 – Saída do comando ethtool eth0	38
Figura 24 – Saída do comando ethtool eth1	38
Figura 25 – Saída do comando ethtool lo.....	38
Figura 26 – Saída do comando netdiscover	39
Figura 27 – Saída do comando netstat	40
Figura 28 – Saída do comando route	40
Figura 29 – Saída do comando traceroute	41
Figura 30 – Tela do Wireshark com pacotes da captura Analise_02.pcap.....	42
Figura 31 – Entrada da tabela ARP do host B.....	43
Figura 32 – Entrada da tabela ARP do host D	44
Figura 33 – TCP STREAM <i>Wireshark</i>	45
Figura 34 – Saída do comando utilizado para instalação do Foremost.....	46
Figura 35 – Saída do comando utilizado para realizar a instalação do TCPFlow.....	46
Figura 36 – Diagrama do ambiente de realização da captura cap_01.pcap	47
Figura 37 – Arquivos extraídos no pcap.....	50
Figura 38 – Arquivos .jpg extraídos no processo	50
Figura 39 – Arquivos .pdf extraídos no processo	50
Figura 40 – Arquivo de planta baixa capturado	51
Figura 41 – Arquivos .png recuperados do pcap.....	51
Figura 42 – Imagem de projeto residencial obtido na extração	52
Figura 43 – Diagrama do ambiente de realização da captura cap_01.pcap	53
Figura 44 – Imagem da ferramenta <i>NetworkMiner</i>	55
Figura 45 – Imagem da ferramenta <i>NetworkMiner</i>	55
Figura 46 – Arquivo 2quartos.pdf	56
Figura 47 – Arquivo 2quartos.pdf	56

LISTA DE QUADROS

Quadro 1 – Especificações da máquina virtual	13
Quadro 2 – Especificações da Máquina Virtual Kali.....	35
Quadro 3 – Quadro de comandos para verificação das informações da rede .	36
Quadro 4 – Organização dos dados da observação do tráfego do Wireshark .	43
Quadro 5 – Comandos para extração de arquivos do cap_01.pcap	48

EMANIP

LISTAGEM DE COMANDOS

Comando de prompt 1 – Instalação do servidor SSH	14
Comando de prompt 2 – Ações do insider	16
Comando de prompt 3 – Instalação de pacotes adicionais	17
Comando de prompt 4 – Instalação do <i>Volatility</i>	18
Comando de prompt 5 – Comandos para preparação do LKM	22
Comando de prompt 6 – Transferência e carga do LKM.....	23
Comando de prompt 7 – Execução do dump	23
Comando de prompt 8 – Extração dos <i>hashes</i>	24
Comando de prompt 9 – Evidências extraídas.....	25
Comando de prompt 10 – Evidências extraídas.....	25
Comando de prompt 11 – Obtenção de dados do dispositivo de rede.....	37
Comando de prompt 12 – Comando para obter informações da interface solicitada	37
Comando de prompt 13 – Comando para escaneamento de hosts ativos na rede	39
Comando de prompt 14 – Comando para verificar conexões estabelecidas ...	39
Comando de prompt 15 – Comando para verificar a tabela de rotas	40
Comando de prompt 16 – Comando para verificar a rota de um pacote de rede.....	41
Comando de prompt 17 – Instalação do Foremost	46
Comando de prompt 18 – Instalação do TCPFlow.....	46
Comando de prompt 19 – Extração de conteúdo com o Foremost e TCPFlow	49
Comando de prompt 20 – Instalação do Mono e NetworkMiner.....	53
Comando de prompt 21 – Comando utilizado acessar o diretório do arquivo cap_01.pcap.....	54

SUMÁRIO

PERÍCIA FORENSE EM SISTEMAS IN VIVO - II	6
1 <i>INSIDERS</i>	6
2 DESENVOLVENDO UM EXEMPLO PRÁTICO	7
2.1 Cenário	7
2.2 Planejando o processo pericial	7
2.2.1 <i>Information governance</i>	8
2.2.2 <i>Identification</i>	9
2.2.3 <i>Preservation</i>	10
2.2.4 <i>Collection</i>	10
2.2.5 <i>Processing</i>	11
2.2.6 <i>Review</i>	11
2.2.7 <i>Analysis</i>	12
2.2.8 <i>Production</i>	12
2.2.9 <i>Presentation</i>	12
2.2.10 Representação do processo pericial	12
3 HANDS-ON: ANÁLISE FORENSE DE <i>FILE SERVER</i>	13
3.1 Ambiente	13
3.2 O <i>insider</i> em ação	15
4 COMEÇANDO O PROCESSO INVESTIGATIVO	16
4.1 Desenvolvendo a análise forense da RAM	16
4.1.1 Identificando o ambiente do objeto questionado	16
4.2 Implementando o ambiente de trabalho	17
4.3 Instalação do <i>Volatility</i> e criação de perfis	18
4.4 Instalação do <i>LiME</i> e criação do LKM	21
4.5 Transferência do LKM e criação do primeiro socket	22
4.6 Carga do LKM e execução do <i>dump</i>	23
4.7 Execução e processamento do <i>dump</i> da RAM	25
4.8 Revisão e análise das evidências	26
4.9 Elaborando um parecer técnico simplificado	29
5 ANÁLISE FORENSE DE TRÁFEGO DE REDES	32
5.1 Conceito e emprego da análise forense de tráfego de redes	32
5.2 Onde capturar o tráfego	33
5.3 Montando o laboratório	35
5.4 Levantamento de dados da rede	35
5.5 Analisando tráfego com o <i>Wireshark</i>	41
5.6 Utilizando as ferramentas <i>Foremost</i> e <i>TCPFLOW</i> para extração de arquivos	45
5.6.1 Instalação do <i>Foremost</i> e <i>TCPFLOW</i>	45
5.6.2 Extraíndo arquivos de uma captura de tráfego com o <i>Foremost</i> e <i>TCPFLOW</i>	47
5.7 Utilização da ferramenta <i>NetworkMiner</i>	52
5.7.1 Instalação	52
5.7.2 Extraíndo arquivos de um <i>pcap</i> com a ferramenta <i>networkminer</i>	53
REFERÊNCIAS	58

PERÍCIA FORENSE EM SISTEMAS IN VIVO - II

1 *INSIDERS*

Insiders podem ser definidos como colaboradores mal-intencionados, com variados graus de acesso e conhecimento dos recursos e da topologia da rede local (LAN) que possam, eventualmente, utilizar seu conhecimento em prejuízo da empresa, por exemplo, praticando ataques internos a aplicações e/ou sistemas, até mesmo revelando informações privilegiadas a terceiros sem a devida autorização. É natural, portanto, que as empresas tenham especial preocupação em relação a esses indivíduos e às implicações de suas ações para a instituição. Diversos fatores podem motivar tal comportamento, essencialmente recaindo sobre a insatisfação com a empresa e obtenção de vantagens (geralmente financeiras).

Em seu trabalho, Gomes (2015) cita pesquisa divulgada pela FIESP na qual apurou-se que 23,5% das empresas entrevistadas na época não sabiam dizer se haviam sido alvo de ataques, e desse percentual, 19,5% eram empresas de grande porte, o que permite concluir que as empresas brasileiras, ao menos até então, ainda não se encontravam claramente conscientes em relação a ataques cibernéticos externos.

O que dizer, então, em relação a ataques internos?

A título de exemplo, Gomes (2015) cita o caso de uma empresa paulista que, à época, perdia mais de R\$ 2.250.000,00 diários, totalizando uma perda mensal de R\$ 67.250.000,00, após um de seus funcionários vender o banco de dados da empresa a um concorrente. A solução encontrada pela empresa para tratar e prevenir futuros danos baseou-se na elaboração e difusão de uma cartilha de conduta cibernética, assim como na implementação de sistemas de segurança internos básicos, como a não autorização de e-mails e contas pessoais nos computadores da empresa e o monitoramento das atividades dos funcionários.

Vale destacar que, ao se referir a “colaboradores mal-intencionados”, faz-se referência a funcionários ou ex-funcionários da própria instituição, funcionários terceirizados, parceiros e prestadores de serviços, dentre outros.

Assim sendo, pode-se observar que a perícia forense computacional pode constituir importante ferramenta para esclarecimento de incidentes de segurança e fortalecimento das políticas de segurança da empresa, contribuindo, ainda nesse contexto, para a mitigação de riscos à segurança da informação.

Pronto para ajudar o seu cliente?

2 DESENVOLVENDO UM EXEMPLO PRÁTICO

A fim de aplicar os conceitos estudados até o momento e praticar em uma situação próxima do que poderá encontrar em seu cliente, bem como fazer uso prático de algumas das principais ferramentas voltadas à forense computacional da memória, considere-se o cenário a seguir descrito.

2.1 Cenário

Em determinada empresa, um servidor dedicado ao armazenamento dos arquivos de funcionários encontra-se sob a administração de um funcionário suspeito de acessar tais arquivos sem a devida autorização (*insider*), coletando e repassando clandestinamente a terceiros informações sigilosas provenientes desses documentos. O objetivo do analista forense é emitir um parecer técnico que confirme ou não tal suspeita.

2.2 Planejando o processo pericial

Tratando-se de um trabalho de cunho forense, a utilização de *frameworks* e modelos de referência faz-se necessária, a fim de garantir a correta condução do processo investigativo e, assim, a qualidade e validade dos resultados finais. Assim sendo, toma-se o EDRM como modelo de referência, dada sua íntima relação com o ambiente corporativo (figura “O modelo EDRM”).

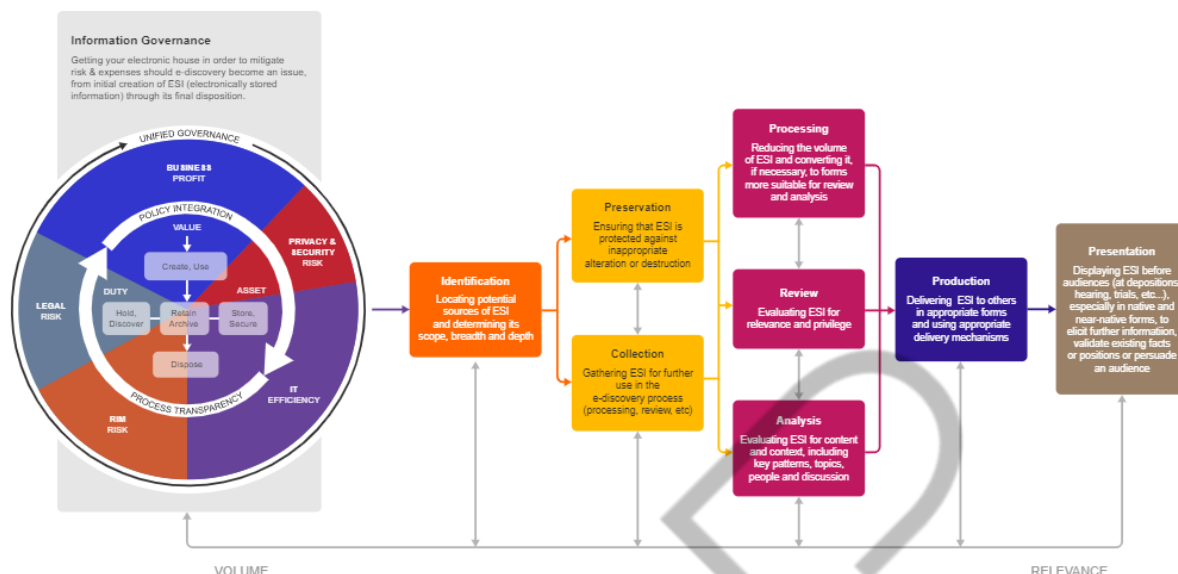


Figura 1 – O modelo EDRM
Fonte: EDRM Model (2019)

Embora o cenário proposto seja uma demanda simples, procedimentos forenses deverão ser adotados na condução dos trabalhos de modo a possibilitar, dentre outros fatores, a determinação da autoria e coleta de evidências que ajudem a comprovar adequadamente eventuais condutas antijurídicas.

2.2.1 Information governance

Considerando-se que os arquivos dos usuários constituem informações eletronicamente armazenadas (*electronically stored information* – ESI), e que o respectivo vazamento pode representar riscos à empresa, dada a quebra de sua confidencialidade, o processo investigativo deverá iniciar o processo de Governança da Informação (*Information governance*), o qual, em linhas gerais, tem como foco a organização do ambiente de produção (TI) da corporação, visando a mitigar riscos e reduzir despesas, abrangendo desde a criação até o descarte da ESI.

Esse processo é consideravelmente complexo, entretanto, por razões de simplicidade, pode ser inicialmente dividido em três categorias:

1. Usuários (corporativos) que necessitam de informações para desempenhar suas funções.
2. Departamentos de TI que devem implementar a mecânica do gerenciamento de informações.
3. Departamentos jurídicos, de risco e reguladores que compreendem o dever da organização de preservar informações além de seu valor comercial imediato.

Nesse contexto, é possível concluir, também, com base no disposto nos itens 1 e 2, que:

Embora sob o aspecto técnico, o administrador do servidor tenha privilégios suficientes para acessar os arquivos (informações) dos usuários armazenados nesse sistema, ele não tem **prerrogativas administrativas** para isso. Assim sendo, o acesso a tal prática caracteriza um incidente de segurança, ao violar a confidencialidade da informação, devendo, portanto, ser devidamente tratado.

2.2.2 Identification

Observadas as considerações pertinentes à governança da informação, faz-se necessária, agora, a identificação de possíveis fontes de ESI relacionadas à investigação. No contexto abordado, isso será limitado à identificação positiva do objeto questionado, consistindo basicamente em:

- Identificar o *host* (*hostname*).
- Identificar a distribuição instalada.
- Identificar o *kernel* em execução (versão, arquitetura etc.).
- Identificar o endereço IP atribuído ao *host*.
- Identificar o processador e disco(s) rígido(s).

Tratando-se de *host* real, informações adicionais como: marca, modelo e fabricante do hardware do equipamento, dentre outras, deverão ser também coletadas.

2.2.3 Preservation

Primeiro processo do terceiro estágio do EDRM, a preservação (*preservation*) da evidência digital apresenta-se como um dos pilares da perícia forense, devendo garantir que a evidência digital não venha a ser destruída, nem que sofra qualquer alteração inapropriada (contaminação da evidência).

Tratando-se, no contexto abordado, de um sistema *in vivo*, a preservação da evidência digital se torna mais difícil, em função, por exemplo, da própria dinâmica do sistema analisado (ARRUDA, 2018). Nesses sistemas, processos estão constantemente sendo iniciados e finalizados, alterando, dessa maneira, o estado do conteúdo da memória, o que torna impossível a obtenção de todas as informações nela armazenadas (FARMER; VENEMA, 2011).

Logo, pode-se concluir que a preservação da evidência digital contida na RAM se dará basicamente, pela redução da interação do analista com o objeto questionado, por exemplo, ao minimizar a execução de comandos (ferramentas), executando essas ferramentas em momento adequado, pela criação de imagem forense de seu conteúdo (simplicadamente aqui também referenciado como *dump* da RAM). Um exemplo disso recai sobre a coleta de informações gerais como: coleta do *hostname* e endereço IP, dentre outras, após a produção da imagem forense.

2.2.4 Collection

Segundo processo do terceiro estágio do EDRM, a coleta (*collection*) da evidência digital, no caso, o conteúdo da memória RAM, o qual será extraído com auxílio do LiME: módulo carregável pelo *kernel* (*loadable kernel module* – LKM), e gravado em forma de arquivo por meio de cópia bit stream, ou seja, a imagem forense da RAM.

A fim de preservar o ambiente do objeto questionado, a preparação (compilação) do LKM deverá ser executada em máquina virtual, fiel às características originais do objeto questionado (distribuição, versão do kernel e arquitetura: 32 ou 64 *bits*), implementada na estação de trabalho do analista.

Observe-se, como exemplo, que se o objeto questionado for um Debian executando um *kernel* original do projeto, qualquer distribuição derivada do Debian (como o Point Linux, dentre outras) que utilize exatamente o mesmo *kernel*, poderá se valer do mesmo LKM. Entretanto, se esse módulo for utilizado em um *host* Fedora, mesmo que com *kernel* de mesma versão e arquitetura, o LKM dificilmente funcionará adequadamente.

A gravação do *dump* também será feita na referida máquina virtual, via rede, uma vez que a gravação no próprio objeto questionado poderá impactar negativamente sobre eventual análise *post mortem*, executada *a posteriori*.

Cabe destacar ainda que:

- 1 Finalizado o *dump* da RAM, o arquivo deverá ter dois *hashes* imediatamente extraídos (um md5 e outro sha1, por exemplo), a fim de possibilitar a verificação da integridade de cópias de trabalho a serem criadas a partir desse arquivo.
- 2 A gravação via rede será executada via sockets TCP criados pelo *netcat*, ferramenta bastante confiável, a qual, entretanto, trafega os dados em *clear text*, o que é altamente impróprio para uma análise forense. Recomenda-se, portanto, que os dados do *netcat* sejam trafegados via SSH.

2.2.5 Processing

Com o auxílio do Volatility, as evidências eletrônicas, por exemplo, processos em execução na memória, *threads* desses processos, conexões de rede ativas e comandos executados, dentre outras, serão extraídas do *dump* e gravadas em arquivos texto, gerando considerável volume de dados.

Primeiro processo do quarto estágio do EDRM, o processamento (*processing*) tem como função filtrar as evidências produzidas, extraindo os dados efetivamente pertinentes e relevantes à investigação, dessa forma, reduzindo o volume de dados a serem analisados e aumentando a sua relevância.

2.2.6 Review

Segundo processo do quarto estágio do EDRM, pode-se entender, de forma bastante simplificada, que a revisão (*REVIEW*) tem como objetivo ajudar a equipe jurídica a entender o valor do conteúdo das evidências (documentos) obtidas, organizando-as, agrupando-as e verificando o quão responsivas essas evidências

(documentos) podem ser para a demanda. Embora seja possível extrair documentos ou, ao menos, fragmentos desses diretamente da memória RAM, esse não será o caso no contexto abordado.

2.2.7 Analysis

Terceiro processo do quarto estágio do EDRM, a análise (*analysis*) das evidências (já filtradas) tem como objetivo permitir ao analista forense avaliar o conteúdo e contexto dessas para responder aos questionamentos formulados, construir a cronologia dos fatos e esclarecer as circunstâncias acerca do incidente de segurança, dentre outras possibilidades.

2.2.8 Production

Processo que constitui o quinto estágio do EDRM, a produção (*production*) consiste, simplificadamente, na elaboração do parecer do analista forense de forma adequada ao entendimento do requisitante. No contexto abordado, neste estágio o analista forense apresentará ainda suas sugestões e considerações para reforço das políticas de segurança da empresa, complementando, assim, a resposta ao incidente.

2.2.9 Presentation

Processo que constitui o sexto estágio do EDRM, na apresentação (*presentation*), o analista forense expõe os resultados de seu trabalho para esclarecer eventuais dúvidas, responder a novos questionamentos e validar fatos ou conclusões, dentre outras possibilidades.

2.2.10 Representação do processo pericial

A Figura “Representação simplificada do processo pericial” apresenta o fluxo das principais tarefas a serem desenvolvidas durante o processo investigativo, simplificadamente.

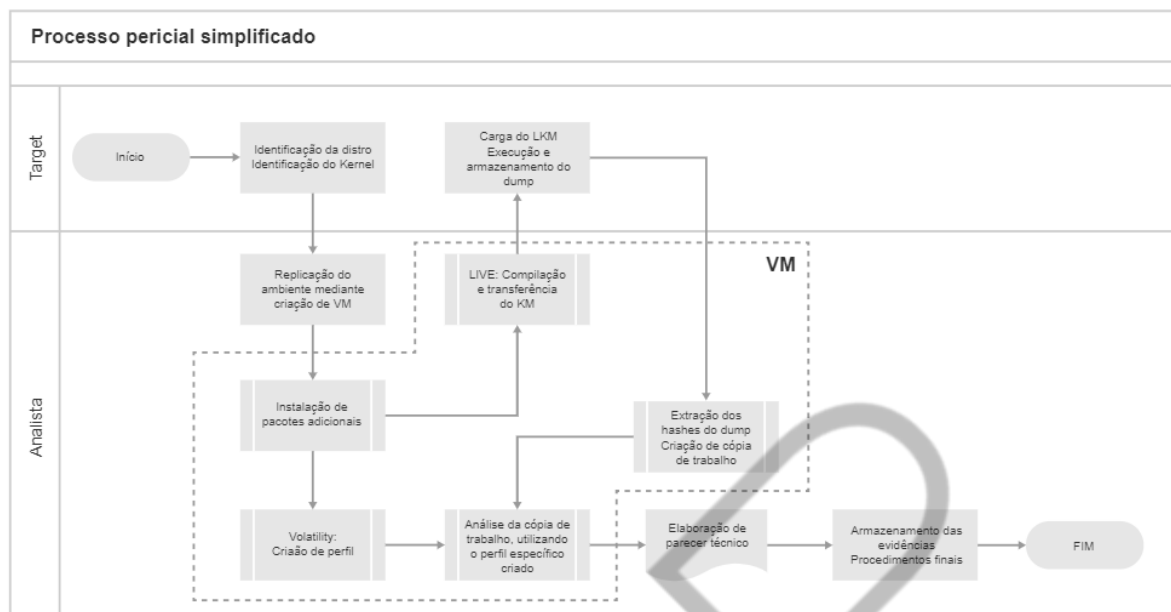


Figura 2 – Representação simplificada do processo pericial
Fonte: Adaptado de Arruda (2018)

3 HANDS-ON: ANÁLISE FORENSE DE *FILE SERVER*

Este *hands-on* tem como objetivo aplicar de forma prática as ferramentas e princípios anteriormente abordados, procedendo à análise forense do conteúdo da RAM, conforme o cenário proposto anteriormente.

3.1 Ambiente

Para desenvolvimento do cenário, serão utilizadas três máquinas virtuais (VMs) conforme especificado no Quadro “Especificações da máquina virtual”.

Parâmetro	Especificação
Hypervisor	VirtualBox
Sistema operacional	Debian 8 - 64bits
Memória	1GB
Disco	8GB
Rede	Rede NAT
vCPU	1

Quadro 1 – Especificações da máquina virtual
Fonte: Elaborado pelo autor (2019)

Tais VMs representarão: (I) estação de trabalho do analista forense (*hostname analista*), O objeto questionado, ou seja, o servidor objeto da investigação (*hostname target*) e (III) A estação de trabalho do *insider* (*hostname insider*), conforme representado na Figura “Topologia geral do cenário”.

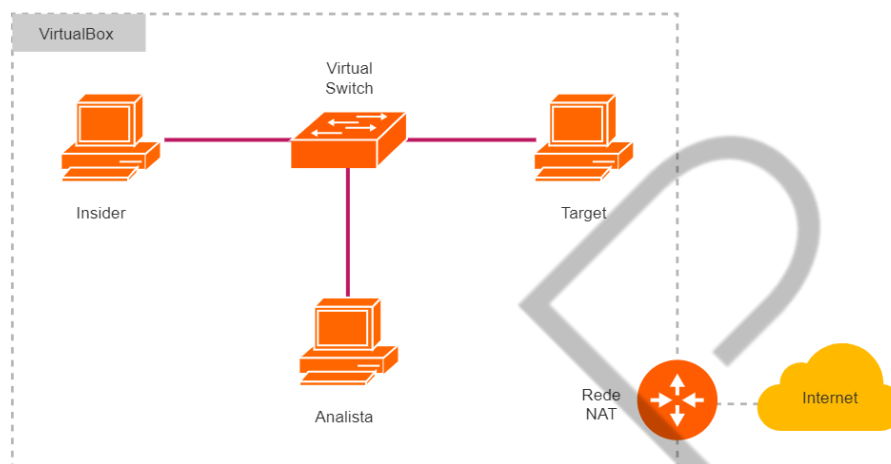


Figura 3 – Topologia geral do cenário
Fonte: Elaborado pelo autor (2019)

Antes de iniciar o processo investigativo, alguns ajustes nos hosts se fazem necessários. Como, para efeito do cenário, o objeto questionado (Target) representa um servidor utilizado para armazenamento de documentos dos usuários, o serviço SSH deverá ser habilitado para permitir a administração remota do referido *host*, a exemplo do geralmente praticado em situações reais. Se o serviço não for instalado durante a criação do *host* (instalação da VM), poderá ser feito posteriormente e mediante à seguinte linha de comando (Listagem “Instalação do servidor SSH”).

```
#atualiza a lista de pacotes que podem ser instalados no
sistema, e se nenhum erro ocorrer (&&), instala o pacote
openssh-server
root@target:~# apt-get update && apt-get install -y openssh-
server
```

Comando de prompt 1 – Instalação do servidor SSH
Fonte: Elaborado pelo autor (2019)

```
root@target:~# systemctl status sshd
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled)
   Active: active (running) since Dom 2019-02-03 13:03:42 -02; 4min 20s ago
   Process: 2988 ExecStartPre=/usr/sbin/sshd -t (code=exit, status=0/SUCCESS)
   Main PID: 2991 (sshd)
   CGroup: /system.slice/ssh.service
           └─ 2991 /usr/sbin/sshd -D

Fev 03 13:03:42 target systemd[1] : Started OpenBSD Secure Shell server.
Fev 03 13:03:42 target sshd[2991] : Server listening on 0.0.0.0 port 22.
Fev 03 13:03:42 target sshd[2991] : Server listening on :: port 22.
root@target:~# █
```

Figura 4 – Verificação do servidor SSH

Fonte: Elaborado pelo autor (2019)

Para verificar se o serviço está devidamente operacional, execute a linha de comando destacada na Figura “Verificação do servidor SSH”, e observe se a resposta obtida aponta o serviço como *Active (running)*.

Para finalizar a preparação do objeto questionado, gere alguns arquivos no *home directory* do usuário regular *user1*, criado durante a instalação da VM.

3.2 O *insider* em ação

Verificar o endereço IP do objeto questionado (*Target*) e, a partir da estação do atacante (*Insider*), executar nele os comandos descritos na Listagem “Ações do Insider”, simulando as ações do *insider* no objeto questionado.

Como, no contexto adotado, o *insider* é o administrador do servidor, admite-se também que este conheça as credenciais dos usuários do servidor, por exemplo, tendo verificado os *usernames* no arquivo */etc/passwd* do servidor e, depois, quebrado as respectivas senhas com auxílio do *John the Ripper* aplicado contra uma cópia do arquivo */etc/shadow*, o que possibilitaria ao atacante acessar os documentos dos usuários de maneira mais furtiva.

Assume-se que o atacante acesse o sistema remoto com as credenciais do usuário cujos documentos pretende acessar, no caso, o usuário *user1*.

```
#insider acessa remotamente o objeto questionando via SSH,
utilizando as credenciais do usuário user1
root@insider:~# ssh user1@10.0.2.6
#insider lista conteúdo do diretório Documentos do usuário
user1
root@target:~#ls Documentos/
```

```
#insider cria arquivo compactado com os arquivos do usuário
user1
root@target:~# tar cvzf docsUser1.tar.gz Documentos/*
#insider copia o arquivo compactado para sua estação de
trabalho e permanece conectado ao objeto questionado
root@target:~# scp docsUser1.tar.gz user1@10.0.2.5:~
```

Comando de prompt 2 – Ações do insider

Fonte: Elaborado pelo autor (2019)

Vale lembrar que os endereços IP em seu cenário poderão ser diferentes dos aqui utilizados.

4 COMEÇANDO O PROCESSO INVESTIGATIVO

4.1 Desenvolvendo a análise forense da RAM

Finalizados os preparativos introdutórios, é possível começar o processo investigativo.

4.1.1 Identificando o ambiente do objeto questionado

Na Figura “Representação simplificada do processo pericial”, observa-se que o processo investigativo será iniciado por meio de acesso interativo ao objeto questionado. Nesse momento, serão coletadas somente as informações necessárias à virtualização do ambiente (criação da VM) do objeto questionado, a fim de produzir o LKM a ser empregado para o *dump* do conteúdo da RAM.

```
root@target:~# cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 8 (jessie)"
NAME="Debian GNU/Linux"
VERSION_ID="8"
VERSION="8 (jessie)"
ID=debian
HOME_URL="http://www.debian.org/"
SUPPORT_URL="http://www.debian.org/support"
BUG_REPORT_URL="http://bug.debian.org/"
root@target:~# uname -r
3.16.0-4-amd64
```

[1]

[2]

root@target:~# █

Figura 5 – Informações do ambiente

Fonte: Elaborado pelo autor (2019)

Na figura “Informações do ambiente” observa-se que o objeto questionado é um *host* Debian 8 – Jessie (destaque 1), executando *kernel* 3.16.0-4 em arquitetura 64 bits (destaque 2).

4.2 Implementando o ambiente de trabalho

Com base nas informações obtidas em 8.4.1, procede-se ao *download* da imagem ISO desse sistema operacional – a partir do *site* original do projeto, implementando-se, então, a VM que replicará o ambiente do objeto questionado (referenciada no cenário como a estação de trabalho do analista).

A implementação do ambiente de trabalho do analista depende da instalação dos seguintes pacotes adicionais:

- 1 *Build-essential*: contém uma lista informativa de pacotes considerados essenciais para a construção de (outros) pacotes no Debian.
- 2 *Linux-headers*: arquivo de cabeçalhos necessários, dentre outras coisas, à compilação do LKM (LiME).
- 3 *Git*: pacote para *download* do LiME a partir do GitHub.
- 4 *Zip*: tradicional empacotador/compressor de arquivos do Linux.

A listagem “Instalação de pacotes adicionais” ilustra o comando para instalação dos referidos pacotes.

```
#atualiza a lista de pacotes que podem ser instalados no
sistema, e se nenhum erro ocorrer (&&), instala os pacotes
especificados
root@analista:~# apt-get update && apt-get install -y build-
essential linux-headers-$(uname -r) git zip
Ign http://ftp.br.debian.org jessie InRelease
Obter:1 http://ftp.br.debian.org jessie-updates InRelease [145
kB]
Atingido http://ftp.br.debian.org jessie Release.gpg
Atingido http://ftp.br.debian.org jessie Release
Atingido http://ftp.br.debian.org jessie-updates/main Sources
Obter:2 http://ftp.br.debian.org jessie-updates/main amd64
Packages/DiffIndex [11,8 kB]
!<--- muitas linhas omitidas --->
```

Comando de prompt 3 – Instalação de pacotes adicionais
Fonte: Elaborado pelo autor (2019)

A utilização da função '\$(uname -r)' ao final do nome do pacote *linux-headers* faz com que a versão do *kernel* corrente seja automaticamente identificada e anexada ao nome do pacote, assegurando a instalação dos *headers* corretos. Dois pacotes adicionais, as ferramentas forenses propriamente ditas, ainda devem ser instaladas.

Sintetizando, a primeira é o [Volatility](#), um *framework open source* desenvolvido em Python especificamente para análise de *dumps* da memória RAM em sistemas *Linux*, *Windows*, *Mac* e *Android*. A segunda é o [LiME](#), abreviação de *Linux Memory Extractor*, um módulo carregável pelo *kernel* (LKM), ou seja, o componente responsável pela execução do *dump* do conteúdo da RAM em sistemas *Linux* e *linux-based*, como o *Android*. Embora ambos possam ser baixados diretamente dos respectivos projetos, opta-se aqui pela aquisição do *Volatility* a partir dos repositórios do *Debian*, a fim de facilitar o processo de instalação.

4.3 Instalação do *Volatility* e criação de perfis

O processo de instalação do *Volatility*, na estação do analista forense, conforme anteriormente descrito, é muito simples, bastando uma única linha de comando (listagem “Instalação do *Volatility*”).

```
#atualiza a lista de pacotes que podem ser instalados no
sistema, e se nenhum erro ocorrer (&&), instala os pacotes que
compõem o Volatility
root@analista:~# apt-get update && apt-get install -y
volatility
```

Comando de prompt 4 – Instalação do *Volatility*
Fonte: Elaborado pelo autor (2019)

Durante o processo de instalação, outros pacotes auxiliares necessários ao *Volatility* serão instalados, dos quais se destacam o *dwarfdump* e o *volatility-tools*. A efetiva utilização do *Volatility* depende da existência de um [perfil](#) específico para o sistema operacional em execução no objeto questionado. Um perfil do *Linux* é essencialmente um arquivo *zip* com informações sobre as estruturas de dados do *kernel* e os símbolos de depuração. Se um perfil preexistente compatível com as características do objeto questionado não estiver disponível, ele terá, então, de ser construído.

- 1) Diversos perfis estão disponíveis e poderão ser baixados do *GitHub*: <https://github.com/volatilityfoundation/profiles>. A Figura “Criação do *vtype*” demonstra a criação do *vtype* (estrutura de dados do *kernel*) a ser empregado pelo perfil em construção.

```

root@analista:~# cd /usr/share/volatility/tools/linux
root@analista:/usr/share/volatility/tools/linux# make
make -C //lib/modules/3.16.0-4-amd64/build CONFIG_DEBUG_INFO=y M="/usr/share/volatility/tools/linux" modules
make[1]: Entering directory '/usr/src/linux-headers-3.16.0-4-amd64'
make[1]: Entering directory '/usr/src/linux-headers-3.16.0-4-amd64'
  CC [M] /usr/share/volatility/tools/linux/module.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC /usr/share/volatility/tools/linux/module.mod.o
  LD [M] /usr/share/volatility/tools/linux/module.ko
make[1]: Leaving directory '/usr/src/linux-headers-3.16.0-4-amd64'
dwarfdump -di module.ko > module.dwarf
make -C //lib/modules/3.16.0-4-amd64/build M="/usr/share/volatility/tools/linux" clean
make[1]: Entering directory '/usr/src/linux-headers-3.16.0-4-amd64'
make[1]: Entering directory '/usr/src/linux-headers-3.16.0-4-amd64'
  CLEAN /usr/share/volatility/tools/linux/.tmp_versions
  CLEAN /usr/share/volatility/tools/linux/Module.symvers
make[1]: Leaving directory '/usr/src/linux-headers-3.16.0-4-amd64'
root@analista:/usr/share/volatility/tools/linux# █

```

Figura 6 – Criação do *vtype*
Fonte: Elaborado pelo autor (2019)

```

root@analista:/usr/share/volatility/tools/linux# ls -l
total 2092
-rw-r--r-- 1 root root 384 Ago 3 2014 Makefile
-rw-r--r-- 1 root root 05571 Ago 3 2014 module.c
-rw-r--r-- 1 root root 2115427 Fev 5 11:46 module.dwarf
drwxr-xr-x 2 root root 4096 Fev 4 19:44 pmem
root@analista:/usr/share/volatility/tools/linux# █

```

Figura 7 – Arquivo *module.dwarf*
Fonte: Elaborado pelo autor (2019)

No destaque na Figura “Criação do *vtype*”, observa-se que a criação do *vtype* resume-se em verificar o código-fonte e compilar o arquivo 'module.c' contra o kernel que se deseja analisar, por intermédio do comando *make*, tendo como resultado a criação do arquivo *module.dwarf* (figura “Arquivo *module.dwarf*”).

```

root@analista:/usr/share/volatility/toosl/linux# zip Debian8 - $(uname -r).zip module
.dwarf /boot/System.map-3.16.0-4-amd64
  adding:  module.dwarf (deflated 91%)
  adding:  boot/System.map-3.16.0-4-amd64 (deflated 79%)
root@analista:/usr/share/volatility/toosl/linux# ls -l
total 2836
-rw-r--r--  1 root root 760875  Fev  5  12:06 Debian8-3.16.0-4-amd64.zip
-rw-r--r--  1 root root   384    Ago  3  2014 Makefile
-rw-r--r--  1 root root  15571  Ago  3  2014 module.c
-rw-r--r--  1 root root 2115427  Fev  5  11:46 module.dwarf
drwxr-xr-x  2 root root   4096  Fev  4  19:44 pnem
root@analista:/usr/share/volatility/toosl/linux# █

```

Figura 8 – Finalização do perfil
Fonte: Elaborado pelo autor (2019)

- 2) A finalização do perfil se dará mediante a criação do arquivo .zip composto pelos arquivos module.dwarf e System.map, mais a sua transferência para o diretório no qual o Volatility armazena os perfis (Figura “Finalização do perfil”).
- 3) A linha ‘zip Debian8-\$(uname -r).zip module.dwarf/boot/System.map-3.16.0-4-amd64’ cria um arquivo formato zip, denominado Debian8-<versão do kernel> (ver destaque na Figura “Finalização do perfil”) a partir dos arquivos module.dwarf e System.map. A transferência do arquivo com o perfil para o diretório padrão do Volatility poderá ser feita por meio do comando ‘mv’, em destaque na Figura “Transferência do perfil”.

```

root@analista:/usr/share/volatility/tools/linux# mv Debian8-3.16.0-4-amd64.zip /usr
/lib/python2.7/dist-packages/volatility/plugins/overlays/linux/
root@analista:/usr/share/volatility/tools/linux# ls -l /usr/lib/python2.7/dist-pack
ages/volatility/plugins/overlays/linux
total 936
-rw-r--r--  1 root root 760875  Fev  5  12:06 Debian8-3.16.0-4-amd64.zip
-rw-r--r--  1 root root  25551  Ago  3  2014 elf.py
-rw-r--r--  1 root root  29704  Fev  4  19:44 elf.py
-rw-r--r--  1 root root    0    Ago  3  2014 __init__.py
-rw-r--r--  1 root root   165    Fev  4  19:44 __init__.pyc
-rw-r--r--  1 root root  61795  Ago  3  2014 linux.py
-rw-r--r--  1 root root  64423  Fev  4  19:44 linux.pyc
root@analista:/usr/share/volatility/tools/linux# █

```

Figura 9 – Transferência do perfil
Fonte: Elaborado pelo autor (2019)

- 4) Observe-se que a via original do Volatility (*volatility/plugins/overlay/linux*), criada quando este é instalado a partir do projeto original, é bem mais curta que a via criada quando ele é instalado a partir do repositório do Debian. Em caso de dúvidas, o comando *find* poderá ser utilizado para localização dos diretórios do

Volatility, por exemplo, digitando-se: `'find / -name volatility'`. Para verificar se o perfil é corretamente reconhecido e localizado pela ferramenta, basta proceder conforme indicado na Figura “Verificando os perfis disponíveis”.

```
root@analista:~# volatility --info | grep Linux
Volatility Foundation Volatility Framework 2.4
linux_banner          - Prints the Linux banner information
linux_varascan         - A shell in the Linux memory image
LinuxDebian8-3 16 0-4-amd64x64 - A Profile for Linux Debian8-3.16.0-4-amd64 x64
root@analista:~# █
```

Figura 10 – Verificando os perfis disponíveis
Fonte: Elaborado pelo autor (2019)

Observe-se que na linha de comando em destaque na Figura “Verificando os perfis disponíveis”, a palavra Linux deve ser iniciada em caixa alta, ainda que a própria ferramenta tenha ajustado o nome do perfil criado para o formato final. Instalada a ferramenta e tendo o perfil para análise do dump devidamente criado, o ambiente de trabalho do analista está quase pronto para uso, faltando apenas a instalação do LiME e criação do LKM.

Sugere-se, também, a criação de um novo diretório, especificamente para armazenamento das evidências produzidas e condução dos trabalhos periciais, por exemplo, com o nome (hostname) do próprio objeto questionado, a fim de facilitar a identificação.

4.4 Instalação do LiME e criação do LKM

Ainda na estação do analista, a partir de um terminal privilegiado (*root*), a instalação dos pacotes e preparação do LiME poderão ser feitas por meio das linhas indicadas na Listagem “Comandos para preparação do LKM”.

```
#baixa o LiME diretamente do GitHub, criando seu diretório de
trabalho
root@analista:~# git clone
https://www.github.com/504ensicsLabs/Lime
```

```
#entra do diretório onde residem os arquivos para compilação
do LiME
root@analista:~# cd Lime/src

#compila o LiME conforme descrito pelo Makefile
root@analista:~/Lime/src# make
```

Comando de prompt 5 – Comandos para preparação do LKM
Fonte: Elaborado pelo autor (2019)

```
root@analista:~# cd Lime/src
root@analista:~/Lime/src# make
make -C /lib/modules/3.16.0-4-amd64/build M="/root/Lime/src" modules
make[1] : Entering directory '/usr/src/linux-headers-3.16.0-4-amd64'
make[1] : Entering directory '/usr/src/linux-headers-3.16.0-4-amd64'
CC [M] /root/Lime/src/tcp.o
CC [M] /root/Lime/src/disk.o
CC [M] /root/Lime/src/main.o
CC [M] /root/Lime/src/hash.o
LD [M] /root/Lime/src/lime.o
Building modules, stage 2.
MODPOST 1 modules
CC /root/Lime/src/lime.mod.o
LD [M] /root/Lime/src/lime.ko
make[1] : Leaving directory '/usr/src/linux-headers-3.16.0-4-amd64'
strip --strip-unneeded lime.ko
mv lime.ko lime-3.16.0-4-amd64.ko
root@analista:~/Lime/src# █
```

Figura 11 – Módulo carregável pelo kernel
Fonte: Elaborado pelo autor (2019)

Se compilado com sucesso, o LKM (arquivo com extensão .ko) já estará pronto para ser carregado para a memória (Figura “Módulo carregável pelo *kernel*”).

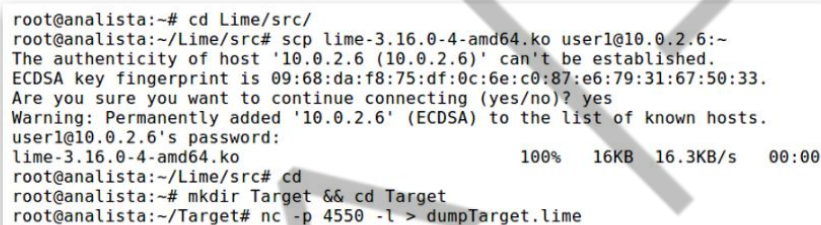
Vale salientar que o módulo (LKM) produzido poderá ser utilizado em qualquer outro sistema *Linux* que execute, exatamente, o mesmo *kernel* sobre o qual o LKM foi compilado.

4.5 Transferência do LKM e criação do primeiro socket

Estando o módulo pronto, ele deverá ser transferido ao objeto questionado e, então, carregado para a RAM para que o *dump* possa ser executado. Uma das maneiras para transferir cópia do LKM para o objeto questionado é por meio do comando SCP, o qual poderá fazer essa cópia com segurança, uma vez que será feita via SSH. O Comando de prompt “Transferência e carga do LKM” descreve a sequência de comandos utilizada na operação, complementada pelo *socket* a ser criado na estação do analista a fim de receber o *dump* da RAM do objeto questionado via rede.

```
#copia o LKM para o home directory de user1 no objeto
questionado, via SCP
root@analista:~/Lime/src# scp lime-3.16.0-4-amd64.ko
user1@10.0.2.6:~
#retorna à raiz do diretório
root@analista:~/Lime/src# cd
#cria e acessa o subdiretório Target
root@analista:~# mkdir Target && cd Target
#com auxílio do netcat, cria um socket colocando a porta 4550
em estado de listen, e direcionando o tráfego recebido através
desta para um arquivo denominado dumpTarget.lime
root@analista:~/Target# nc -p 4550 -l > dumpTarget.lime
```

Comando de prompt 6 – Transferência e carga do LKM
Fonte: Elaborado pelo autor (2019)



```
root@analista:~# cd Lime/src/
root@analista:~/Lime/src# scp lime-3.16.0-4-amd64.ko user1@10.0.2.6:~
The authenticity of host '10.0.2.6 (10.0.2.6)' can't be established.
ECDSA key fingerprint is 09:68:da:f8:75:df:0c:6e:c0:87:e6:79:31:67:50:33.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.0.2.6' (ECDSA) to the list of known hosts.
user1@10.0.2.6's password:
lime-3.16.0-4-amd64.ko                                100% 16KB 16.3KB/s 00:00
root@analista:~/Lime/src# cd
root@analista:~# mkdir Target && cd Target
root@analista:~/Target# nc -p 4550 -l > dumpTarget.lime
```

Figura 12 – Criação do primeiro socket
Fonte: Elaborado pelo autor (2019)

Essa sequência de comandos produzirá a saída exibida na Figura “Criação do primeiro *socket*”.

Observe, pelo destaque na Figura “Criação do primeiro *socket*”, que o *prompt* na estação do analista fica “preso”, aguardando a transmissão dos dados.

4.6 Carga do LKM e execução do *dump*

De volta ao objeto questionado, ainda como root, o LKM poderá ser carregado para a RAM, conforme o Comando de prompt “Execução do *dump*”.

```
#carrega o LKM para a RAM
root@target:~# insmod /home/user1/lime-3.16.0-4-amd64.ko
"path=tcp:4550 format=lime"
#cria o terceiro socket, destinado à interligação da estação
do analista com o objeto questionado a partir DE NOVA ABA NO
TERMINAL. Este socket se fechará automaticamente 3 segundos
após a finalização do dump, fechando assim, também o socket na
estação do analista
root@target:~# nc localhost 4550 | nc 10.0.2.7 4550 -q 3
```

Comando de prompt 7 – Execução do *dump*

Fonte: Elaborado pelo autor (2019)

Da linha de comando no Comando de prompt “Execução do *dump*”, tem-se:

- **insmod lime-3.16.0-4-amd64.ko:** carrega o LKM para a RAM por intermédio do comando *insmod*.
- **path=tcp:4550:** cria um *socket* na porta 4550 através da qual o módulo enviará o *dump* (embora todos os *sockets* tenham sido estabelecidos na porta 4550, isto se deu por simples comodidade, não por necessidade).
- **format=lime:** o arquivo de *dump* será gerado em formato lime.

Conforme anteriormente já citado, tratando-se de análises forenses, é imprescindível que o *dump* produzido tenha dois *hashes* distintos extraídos a fim de que se possa verificar a integridade de futuras cópias de trabalho.

Diferentemente dos *hashes* de senhas, como os *hashes* do *dump* não são sujeitos a ataques por força bruta, sendo, então, extraídos *hash* um md5 e outro sha1 (embora esses já sejam considerados inseguros). O Comando de prompt “Extração dos *hashes*” descreve o procedimento para extração e preservação dos *hashes*.

```
#extraí hash md5 e armazena em arquivo texto denominado
dumpTarget.hashes
root@analista:~/Target# md5sum dumpTarget.lime >
dumpTarget.hashes
#extraí hash sha1 e ADICIONA ao arquivo dumpTarget.hashes
#repara na substituição do redirecionador > por >>, se este
detalhe for #ignorado, o conteúdo original do arquivo será
sobrescrito pelos novos dados #produzidos pela ferramenta
shasum
root@analista:~/Target# shasum dumpTarget.lime >>
dumpTarget.hashes
```

Comando de prompt 8 – Extração dos *hashes*
Fonte: Elaborado pelo autor (2019)

A Figura “*Hashes do dump*” ilustra o conteúdo do arquivo *dumpTarget.hashes*.

```
root@analista:~/Target# cat dumpTarget.hashes
c84e9ef62d1ad71d7345bbe50ecb8b99 dumpTarget.lime
617a49c9eb9c3250ac177bc892aa295dbfc397cd dumpTarget.lime
root@analista:~/Target# █
```

Figura 13 – *Hashes do dump*
Fonte: Elaborado pelo autor (2019)

Finalizado o *dump*, o analista poderá coletar e preservar as informações adicionais necessárias à identificação do objeto questionado e ao eventual preenchimento da cadeia de custódia.

4.7 Execução e processamento do *dump* da RAM

Antes de iniciar seus trabalhos, o analista deverá proceder à cópia do *dump*, extraíndo os seus *hashes* e comparando-os aos *hashes* do *dump* original. Os *hashes* do original e da cópia deverão coincidir perfeitamente.

O analista jamais deverá trabalhar diretamente com o *dump* original.

Com ajuda do *Volatility*, as seguintes evidências serão extraídas da cópia de trabalho do *dump* (Listagem “Evidências extraídas”):

```
#listagem dos comandos executados no objeto questionado ainda
persistentes na RAM quando da execução do dump
root@analista:~/Target# volatility -f dumpTarget.lime --
profile=LinuxDebian8-3_16_0-4-amd64x64 linux_bash > bash.txt
#listagem dos processos em execução no objeto questionado
quando da execução do dump
root@analista:~/Target# volatility -f dumpTarget.lime --
profile=LinuxDebian8-3_16_0-4-amd64x64 linux_psaux
#listagem das threads identificadas na RAM quando do objeto
questionado quando da execução do dump
root@analista:~/Target# volatility -f dumpTarget.lime --
profile=LinuxDebian8-3_16_0-4-amd64x64 linux_threads
#listagem das conexões de rede ativas no objeto questionado
quando do objeto questionado quando da execução do dump
root@analista:~/Target# volatility -f dumpTarget.lime --
profile=LinuxDebian8-3_16_0-4-amd64x64 linux_netstat >
netstat.txt
#extração de strings do dump
root@analista:~/Target# strings -a dumpTarget.lime >
strings.txt
```

Comando de prompt 9 – Evidências extraídas
Fonte: Elaborado pelo autor (2019)

A fim de melhor compreender a utilização do *Volatility*, vamos analisar, a título de exemplo, a primeira linha de comando utilizada na Listagem “Evidências extraídas”:

```
volatility -f dumpTarget.lime --profile=LinuxDebian8-3_16_0-4-
amd64x64 linux_bash > bash.txt
```

Comando de prompt 10 – Evidências extraídas
Fonte: Elaborado pelo autor (2019)

A opção **-f dumpTarget.lime** instrui a ferramenta a analisar o arquivo **dumpTarget.lime**, utilizando o perfil (**--profile**) **LinuxDebian8-3_16_0-4-amd64x64** (previamente criado de acordo com as informações obtidas no objeto questionado).

O **plugin linux_bash** lista os comandos executados no objeto questionado, apurados durante a execução do **dump**. Na última parte dessa linha de comando, tem-se o redirecionador **'>'**, o qual envia a saída produzida para um arquivo chamado **bash.txt**, ao invés da tela, a fim de preservar as evidências obtidas. O **Volatility** dispõe de diversos **plug-ins**, destinados às mais diversas tarefas. Uma listagem desses pode ser obtida com a linha de comando em destaque na Figura “Listagem parcial de **plug-ins**”.

```
root@analista:~/Target# volatility -i -f dumpTarget.lime | grep linux
Volatility Foundation Volatility Framework 2.4
linux_apihooks          - Checks for userland apihooks
linux_arp                - Print the ARP table
linux_banner            - Prints the Linux banner information
linux_bash              - Recover bash history from bash process memory
linux_bash_env          - Recover bash's environment variables
linux_bash_hash         - Recover bash hash table from bash process memory
linux_check_afinfo      - Verifies the operation function pointers of network protocols
linux_check_creds       - Checks if any processes are sharing credential structures
linux_check_evt_arm     - Checks the Exception Vector Table to look for syscall table ho
oking
linux_check_fop          - Check file operation structures for rootkit modifications
linux_check_idt         - Checks if the IDT has been altered
linux_check_inline_kernel - Check for inline kernel hooks
linux_check_modules     - Compares module list to sysfs info, if available
linux_check_syscall     - Checks if the system call table has been altered
linux_check_syscall_arm - Checks if the system call table has been altered
linux_check_tty         - Checks tty devices for hooks
linux_cpuinfo           - Prints info about each active processor
linux_dentry_cache      - Gather files from the dentry cache
linux_dmesg             - Gather dmesg buffer
linux_dump_map          - Writes selected memory mappings to disk
linux_elfs              - Find ELF binaries in process mappings
linux_enumerate_files   - List files referenced by the filesystem cache
linux_find_file         - Lists and recovers files from memory
linux_hidden_modules    - Carves memory to find hidden kernel modules
```

Figura 14 – Listagem parcial de **plug-ins**

Fonte: Elaborado pelo autor (2019)

4.8 Revisão e análise das evidências

As evidências extraídas após o processamento inicial do **dump** da RAM do objeto questionado (8.4.7) deverão ser agora devidamente revisadas e analisadas.

Embora o processo de revisão (*REVIEW*) do EDRM seja altamente relevante em demandas em que haja litígio, em função disso exigindo apurada análise documental, no cenário simples proposto, ele tem pouco destaque, permitindo ao analista passar rapidamente ao processo de análise (*ANALYSIS*) das evidências. A Figura “Arquivo Bash2.txt” ilustra o conteúdo arquivo *bash.txt*, após devidamente processado, expurgando-se os comandos sem relação com a investigação em andamento. Vale observar que o arquivo *bash.txt* originalmente produzido na fase anterior foi preservado, tendo sido salvo o arquivo processado com o nome de Bash2.txt.

```
root@analista:~/Target# head -n1 psaux.txt > psaux2.txt
root@analista:~/Target# grep 1925 psaux.txt >> psaux2.txt
root@analista:~/Target# grep 2250 psaux.txt >> psaux2.txt
root@analista:~/Target# grep 2286 psaux.txt >> psaux2.txt
root@analista:~/Target# cat psaux2.txt
Pid      Uid      Gid      Arguments
1925     1000     1000     -bash
2250      0        0        /bin/bash
2286      0        0        /bin/bash
root@analista:~/Target#
```

Figura 15 – Arquivo Bash2.txt
Fonte: Elaborado pelo autor (2019)

Pid	Name	Command Time	Command
1925	bash	2019-02-05 23:45:45 UTC+0000	ls Documentos/
1925	bash	2019-02-05 23:46:10 UTC+0000	tar cvzf docsUser1.tar.gz Documentos/*
1925	bash	2019-02-05 23:46:26 UTC+0000	scp docsUser1.tar.gz user1@10.0.2.5:~
2250	bash	2019-02-05 23:47:06 UTC+0000	cat /etc/os-release
2250	bash	2019-02-05 23:47:11 UTC+0000	uname -r
2250	bash	2019-02-05 23:48:42 UTC+0000	insmod /home/user1/lime-3.16.0-4-amd64.ko "path=tcp:4550 format=lime"
2286	bash	2019-02-05 23:49:26 UTC+0000	nc localhost 4550 nc 10.0.2.7 4550 -q3

Figura 16 – Arquivo Psaux2.txt
Fonte: Elaborado pelo autor (2019)

No arquivo Bash2.txt, é possível observar que três diferentes processos relacionados à investigação estavam ativos por ocasião do *dump* da RAM: 1925, 2250 e 2286. Processando-se o arquivo *psaux.txt* e salvando-se o resultado sob o nome de *Psaux2.txt*, obtém-se o arquivo apresentado na Figura “Arquivo Psaux2.txt”.

Na primeira linha da Figura “Arquivo Psaux2.txt”, o comando executado extrai a primeira linha (*head -n1*) do início do arquivo *psaux.txt*, redirecionando-a (>) a um arquivo chamado *psaux2.txt*. Nas três linhas subsequentes, os PIDs dos processos (1925, 2250 e 2286, respectivamente) são pesquisados no arquivo *psaux.txt*, com auxílio da ferramenta *grep*. Mais uma vez, destaque-se aqui o uso de redirecionadores duplos (>>), a fim de garantir que o resultado de cada pesquisa seja agregado ao final do arquivo *psaux2.txt*.

Caso um redirecionador simples fosse utilizado (>), o resultado de cada pesquisa sobrescreveria o conteúdo do arquivo, arquivando-se apenas o resultado da última pesquisa, comprometendo, dessa forma, a evidência.

Na Figura “Arquivo Psaux2.txt”, ao listar-se o conteúdo do arquivo, é possível observar que apenas dois usuários estavam logados no objeto questionado na ocasião: o *root* (Uid 0 ; Gid 0), e outro usuário cujos Uid e Gid são 1000.

Processando-se o arquivo *strings.txt* com base na *string* de busca ‘1000:1000’ (formato como o *Linux* armazena o Uid e o Gid do usuário no arquivo */etc/passwd*), obtém-se o resultado preservado no arquivo *id_user.txt* (Figura “Arquivo Id_User.txt”).

```
root@analista:~/Target# grep '1000:1000' strings.txt | tee id_user.txt
1000 : 1000 : user1,,
1000 : 1000 : user1,,
!<- -várias linhas omitidas- ->
1000 : 1000 : user,,
user1:x:1000:1000:user1,,:/home/user1:/bin/bash
1000 : 1000 : user1,,
root@analista:~/Target#
```

Figura 17 – Arquivo Id_User.txt
Fonte: Elaborado pelo autor (2019)

Na linha de comando da Figura “Arquivo Id_User.txt”, a ferramenta *grep* pesquisa a *strings* 1000:1000 no arquivo *strings.txt*. O *pipe* (|) ao final do comando faz com que o resultado da busca seja exibido na tela e também gravado no arquivo *id_user.txt*. Do destaque da Figura “Arquivo Id_User.txt” é possível observar que o usuário designado pelo Uid 1000 e Gid 1000 é o *user1*. Ao pesquisar os processos pertencentes ao usuário *user1*, conforme destacado na Figura “Arquivo Psaux3.txt”, observa-se que a sessão do *user1* era remota, via SSH, em pseudoterminal (pts/0).

```
root@analista:~/Target# grep user1 psaux.txt | tee psaux3.txt
1918    0      0      sshd: user1 [priv]
1924   1000   1000    sshd: user1@pts/0
2283    0      0      insmod /home/user1/lime-3.16.0-4-amd64.ko path=tcp:4550 format=lime
```

Figura 18 – Arquivo Psaux3.txt
Fonte: Elaborado pelo autor (2019)

```
root@analista:~/Target# grep 1924 netstat.txt | tee netstat2.txt
TCP        10.0.2.6      : 22 10.0.2.5      :47128 ESTABLISHED      sshd/1924
UNIX 17645             sshd/1924
UNIX 17646             sshd/1924
UNIX 17664             sshd/1924
```

Figura 19 – Arquivo Netstat2.txt
Fonte: Elaborado pelo autor (2019)

Ao pesquisar no arquivo *netstat.txt* pelo PID 1924 (em destaque na Figura “Arquivo Psaux3.txt”), verifica-se, por meio do arquivo *netstat2.txt*, criado pela linha de comando ilustrada na Figura “Arquivo Netstat2.txt”, que o endereço de origem da conexão é 10.0.2.5:47128.

De posse do endereço de origem da conexão, basta apurar a qual *host* ele está vinculado.

4.9 Elaborando um parecer técnico simplificado

Como no contexto estudado, o processo de revisão (*REVIEW*) do EDRM tem pouca relevância, finalizada a análise das evidências, o analista poderá elaborar seu parecer técnico.

A produção desse parecer, assim como a de um laudo pericial real e completo, deve primar pela objetividade, clareza e simplicidade em sua narrativa, embora essa deva valer-se de linguagem formal.

Exageros no uso do jargão técnico e narrativa prolixa, dentre outros, são exemplos de práticas que devem ser evitadas, assim como o uso de imagens deve ser acompanhado dos devidos comentários e explicações, evitando que o leitor tenha de tirar conclusões a respeito do significado de tais elementos.

O analista forense deve ter sempre em mente a necessidade de se fazer entender clara e inequivocamente, tendo em mente o perfil de quem usufruirá de seu trabalho, portanto, expressando-se adequadamente. Finalmente, a apresentação do trabalho também deverá ser planejada em alinhamento com o perfil demandante e com os objetivos do trabalho.

Uma recomendação geral, com base na observação de trabalhos reais, é que a primeira parte do laudo contenha os elementos textuais gerais, adequadamente organizados, por exemplo, iniciando-se com uma breve introdução acerca do

incidente, descrição do objetivo do trabalho, identificação do perito, questionamentos efetuados e respectivas respostas, dentre outros elementos.

Sugere-se, ainda, que os procedimentos e elementos técnicos, respectivamente, assim como a análise das evidências e suas eventuais cópias (por exemplo, *prints* de tela), sejam integrados ao documento na forma de apêndices.

PARECER TÉCNICO SIMPLIFICADO

Objeto questionado:

File server *hostname* Target, S.O. Debian 8 Jessie, 1GB RAM, <marca, modelo, HD, núm. série, ...>

Objetivo da investigação:

Confirmação de hipótese referente ao acesso remoto não autorizado a documentos de usuários armazenados no objeto questionado, fruto de possível ação de *insider(s)*.

Resumo dos fatos:

Com base na análise das evidências apuradas no objeto questionado, é possível afirmar:

- No dia 05/02/2019, um usuário cuja real identidade ainda não pode ser determinada, valeu-se da credencial ***user1*** para efetuar um acesso remoto ao objeto questionado, **a partir da estação de trabalho do administrador**, fora do horário de expediente normal da empresa.
- As ações executadas por esse usuário, cuja cronologia pode ser observada na TIMELINE das AÇÕES (Apêndice A) verificadas no objeto questionado **sugerem acesso ilegítimo** aos documentos copiados, implicando, assim, na quebra de confidencialidade (“vazamento de informações”), conforme definido pelas políticas de segurança da empresa, caso tal hipótese venha a ser comprovada.

Conclusão e recomendações:

A análise e contextualização das evidências coletadas oferecem sólidos indícios de desvio de finalidade na utilização do serviço de acesso remoto (SSH), sugerindo haver sido, este, utilizado como meio para possibilitar a coleta de documentos de usuários de forma ilegítima. Assim sendo, sugere-se o aprofundamento do processo investigativo, com auxílio do pessoal de segurança física e patrimonial, a fim de determinar-se a verdadeira identidade do autor de tais atos, viabilizando, assim, a efetivação das medidas cabíveis.

São Paulo, 6 de fevereiro de 2019.

<Nome do analista>

Apêndice A

TIMELINE das AÇÕES

Apêndice A

TIMELINE DAS AÇÕES

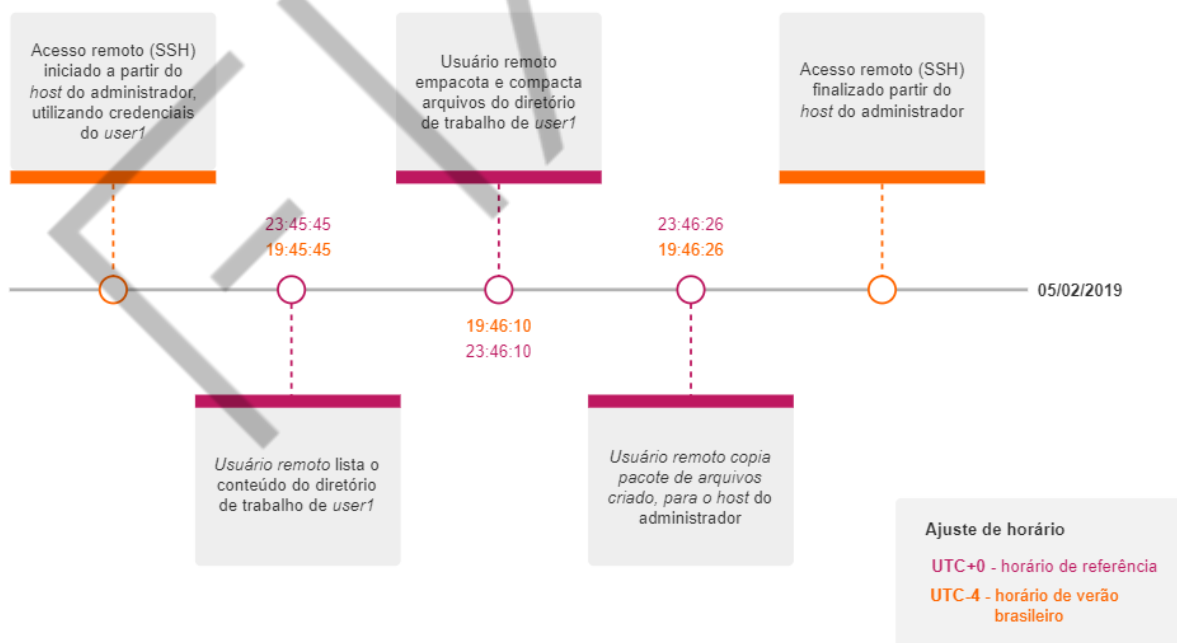


Figura 20 – Timeline das ações
Fonte: Elaborado pelo autor (2019)

Fim do Parecer Técnico Simplificado.

Vale reforçar

A aplicação do EDRM ao cenário proposto exigiu adaptações e simplificações, as quais podem levar a interpretações incorretas ou superficiais desse *framework*. Logo, recomenda-se enfaticamente que se recorra à documentação disponível no *site* do projeto a fim de estudá-lo com a devida profundidade.

5 ANÁLISE FORENSE DE TRÁFEGO DE REDES

5.1 Conceito e emprego da análise forense de tráfego de redes

Dando continuidade ao processo de aprendizagem de uma análise viva, verificamos a necessidade de compreendermos e explicarmos um breve conceito de Análise Forense de Tráfego de Redes, tendo em vista a extrema necessidade de entendermos e levantarmos informações de uma rede.

Com o desenvolvimento das tecnologias de interconexão, quase tudo o que é digital, eletrônico, atualmente está conectado a alguma rede. Na indústria, relês, sensores, atuadores, entre outros componentes eletrônicos encontram-se conectados a uma rede *TCP/IP*, que na maioria dos casos é a própria do escritório. Nas residências existem roteadores, televisores *smart* e centrais inteligentes que controlam lâmpadas, portões e câmeras, e tudo se encontra ligado à mesma rede.

As facilidades de intercomunicação entre esses dispositivos trouxeram inúmeros benefícios e riscos consequentes.

Ser alvo de inúmeras formas de ataques para obtenção de dados sigilosos, sabotagem industrial, *insiders*, entre outros, tornou-se rotineiro. Paralelamente a isso, a análise de tráfego surge como uma das principais formas de monitorar e, possivelmente, identificar incidentes.

Como já é de conhecimento, tráfego de redes consiste na quantidade de dados transmitidos entre dois pontos de uma rede e, conforme Nunes (2017, p. 3), “a Computação Forense tem por objetivo auxiliar na solução de casos em que são cometidos crimes com o auxílio de dispositivos computacionais”. Diante dessas definições, durante a fase de análise de artefatos, verifica-se que tráfegos de rede podem conter elementos que venham a contribuir sobremaneira em um caso pericial, por meio desses, conseguimos extrair informações, arquivos e dados que ajudam na estruturação de uma linha investigativa.

A correta interpretação de um possível tráfego malicioso pode trazer indícios e até conclusões do envolvimento, direta ou indiretamente, de indivíduos e ou equipamentos. Além de ser um ótimo termômetro para terminar corretamente problemas com uma rede.

Sendo assim, para a realização de uma Análise Forense de Tráfego de Redes, faz-se necessário o uso de ferramentas específicas. Dentre várias, abordaremos: *Wireshark*, *Foremost*, *TCPFlow* e *NetworkMiner*.

5.2 Onde capturar o tráfego

Para realizarmos uma captura de tráfego, cujo objetivo principal é a análise forense, a nossa posição na estrutura da rede é de fundamental importância.

Não é em qualquer posição de uma rede que é possível capturar um determinado tráfego, na maioria das vezes, obtemos apenas mensagens de *Broadcast*.

Atualmente, redes são construídas basicamente por dois equipamentos: roteadores e *switches*.

Levando em consideração os nós da rede, ou seja, o que temos como equipamentos intermediários, devemos tomar a melhor decisão de onde capturar um tráfego na rede.

Para que a captura seja efetiva, não importando o *software* que seja usado, devemos nos certificar de que a placa está em *modo promíscuo*, ou seja, todos os pacotes que são vistos pela interface de rede são acessados por ela e, para isso, precisamos ter bem definida a diferença entre as redes com *switches* ou *hubs*.

Cabe lembrar que em uma rede baseada em *switches*, o pacote é enviado exatamente para o destinatário por meio do seu endereço *MAC*, dentro da mesma sub-rede.

Já em uma rede baseada em *hubs*, o pacote é enviado para todos da rede e apenas o destinatário responde. Nesse caso, todos escutam o tráfego.

Após entender como o *hub* se comporta em relação aos tráfegos da rede, podemos perceber que, apesar de não ser muito usual ou comum a utilização desses

equipamentos, eles são de grande utilidade para o trabalho de Análise de Tráfego de rede, o qual podemos utilizar para analisar tudo o que está entrando e saindo de um determinado equipamento na rede.

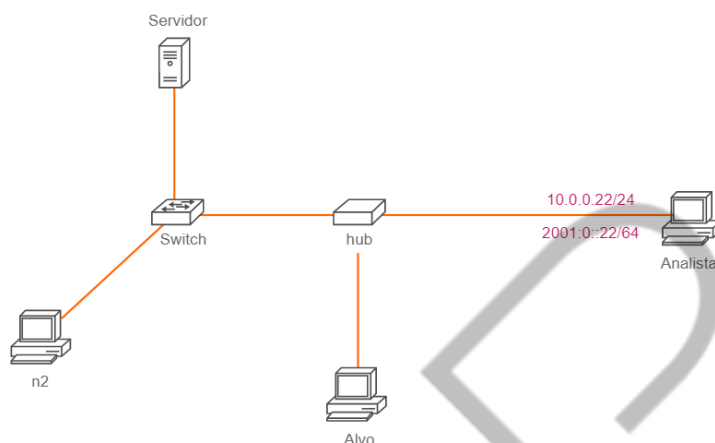


Figura 21 – Posicionamento para captura
Fonte: Elaborado pelo autor (2019)

Como na Figura “Posicionamento para captura”, colocar um hub entre o switch e o alvo nos garante que interceptaremos tudo o que for enviado ou recebido pelo alvo. Ao passo que o analisista, posicionando-se conectado ao mesmo switch que o alvo, garante apenas que serão capturados tráfegos de Broadcast.

Outra opção é realizar a captura a partir do próprio *host* do qual se quer obter o tráfego, por exemplo, instalando e executando o *Wireshark* no próprio *host*, porém, nem sempre poderemos ter acesso ao “alvo”.

Para o caso de o analisista ser ou ter poderes de administrador na rede, ele pode configurar o *switch* para redirecionar o tráfego para uma determinada porta na qual haverá uma máquina preparada para fazer a captura.

O comportamento do tráfego em redes virtuais: conforme Weidman (2016), redes virtuais parecem agir com *hubs* porque todas as máquinas virtuais compartilham um dispositivo físico. Se o tráfego for capturado em modo promíscoo em uma rede virtual, você poderá ver o tráfego de todas as máquinas virtuais, bem como o do computador *host*.

5.3 Montando o laboratório

Para realizarmos os estudos práticos de Análise Forense de Tráfego de Redes, utilizaremos o Sistema Operacional *Kali Linux*, que possui, instaladas e prontas para uso, as principais ferramentas.

O material que será utilizado na execução dos laboratórios encontra-se disponível no endereço: <https://drive.google.com/open?id=1RGryoM6kjKLHW1VNQ_7kqHnF-oQFYar>.

A imagem do sistema operacional encontra-se disponível em: <https://www.virtualbox.org/wiki/Downloads> e para nosso laboratório utilizaremos a versão 2019.4 64 Bit.

Para instalarmos o *Kali Linux* do laboratório, utilizaremos o virtualizador *VirtualBox* 6.1, disponível em: <<https://www.virtualbox.org/>>. Cabe ressaltar que não abordaremos a instalação do *Kali Linux*, pois não constitui objeto de nosso estudo.

A máquina virtual deverá possuir as configurações, detalhadas no Quadro “Especificações da Máquina Virtual Kali”.

Hypervisor	VirtualBox
Sistema Operacional	Kali Linux - 64 Bits
Memória	2 GB
Disco	40 GB
Rede	Bridge
vCPU	1

Quadro 2 – Especificações da Máquina Virtual Kali
Fonte: Elaborado pelo autor (2019)

5.4 Levantamento de dados da rede

Nesta etapa, realizaremos o levantamento das informações essenciais para identificar o *host* e o ambiente de rede objeto de uma ação maliciosa.

Durante a análise, necessitamos coletar informações sobre a descoberta da rede, configurações e características de hardware de rede, conexões locais

estabelecidas, tabelas de roteamento e tabelas de roteamento entre endereços, tendo-se como foco principal a realização de um desenho da rede.

Para execução deste processo, utilizaremos o “Quadro de comandos para verificação das informações da rede”, conforme detalhado abaixo.

Comandos	Função dos Comandos
netdiscover -r	Comando utilizado para executar um <i>scan</i> da rede, retornando o endereço <i>MAC</i> e o IP de <i>hosts</i> .
ifconfig	Comando utilizado para verificar dados da placa de rede da máquina.
netstat -tupan	Comando utilizado para mostrar as conexões de rede ativas e sem resolver nomes.
route	Comando utilizado para mostrar a tabela de roteamento.
ethtool eth0	Comando executado para verificar as características da interface de rede <i>eth0</i> .
tracert <www.uol.com.br>	Comando utilizado para verificar a rota de meu host até o site <www.uol.com.br>.

Quadro 3 – Quadro de comandos para verificação das informações da rede

Fonte: Elaborado pelo autor (2019)

Para iniciar o processo de levantamento de dados da rede, utilizamos o comando *ifconfig*, no qual obtemos o endereço de IP do *host*, *MAC Address* e outros itens, conforme a Listagem de comandos “Obtenção de dados do dispositivo de rede” e a Figura “Saída do comando ifconfig”.

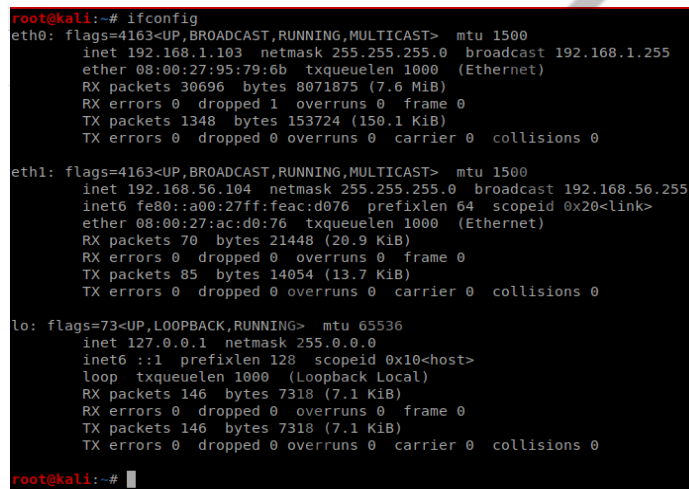
```

root@kali:~# ifconfig
# Comando utilizado para verificar dados do dispositivo de
rede do host.
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.103 netmask 255.255.255.0 broadcast
192.168.1.255
    ether 08:00:27:95:79:6b txqueuelen 1000 (Ethernet)
    RX packets 30696 bytes 8071875 (7.6 MiB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 1348 bytes 153724 (150.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
# 1° Interface da máquina, também chamada de eth0
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.104 netmask 255.255.255.0 broadcast
192.168.56.255
    inet6 fe80::a00:27ff:feac:d076 prefixlen 64 scopeid
0x20<link>
    ether 08:00:27:ac:d0:76 txqueuelen 1000 (Ethernet)
    RX packets 70 bytes 21448 (20.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 85 bytes 14054 (13.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
# 2° Interface configurada na máquina, também chamada de eth1

```

```
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Loopback Local)
    RX packets 146 bytes 7318 (7.1 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 146 bytes 7318 (7.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
# 3º Interface da máquina, também chamada de interface
loopback
```

Comando de prompt 11 – Obtenção de dados do dispositivo de rede
Fonte: Elaborado pelo autor (2019)



```
root@kali:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.103 netmask 255.255.255.0 broadcast 192.168.1.255
    ether 08:00:27:95:79:6b txqueuelen 1000 (Ethernet)
    RX packets 30696 bytes 8071875 (7.6 MiB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 1348 bytes 153724 (150.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.104 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::a00:27ff:feac:d076 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:ac:d0:76 txqueuelen 1000 (Ethernet)
    RX packets 70 bytes 21448 (20.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 85 bytes 14054 (13.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Loopback Local)
    RX packets 146 bytes 7318 (7.1 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 146 bytes 7318 (7.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@kali:~#
```

Figura 22 – Saída do comando ifconfig
Fonte: Elaborado pelo autor (2019)

Para complementar as informações anteriormente coletadas, efetuamos o comando **ethtool**, que retorna os dados técnicos do *hardware* de rede, conforme a Listagem “Comando para obter informações da interface solicitada” e Figuras “Saída do comando ethtool eth0”, “Saída do comando ethtool eth1” e “Saída do comando ethtool lo”.

```
root@kali:~# ethtool eth0
# Comando utilizado para obter informações da interface eth0.
root@kali:~# ethtool eth1
# Comando utilizado para obter informações da interface eth1.
root@kali:~# ethtool lo
# Comando utilizado para obter informações da interface lo.
```

Comando de prompt 12 – Comando para obter informações da interface solicitada
Fonte: Elaborado pelo autor (2019)

```
root@kali:~# ethtool eth0
Settings for eth0:
    Supported ports: [ TP ]
    Supported link modes:   10baseT/Half 10baseT/Full
                           100baseT/Half 100baseT/Full
                           1000baseT/Full
    Supported pause frame use: No
    Supports auto-negotiation: Yes
    Supported FEC modes: Not reported
    Advertised link modes:  10baseT/Half 10baseT/Full
                           100baseT/Half 100baseT/Full
                           1000baseT/Full
    Advertised pause frame use: No
    Advertised auto-negotiation: Yes
    Advertised FEC modes: Not reported
    Speed: 1000Mb/s
    Duplex: Full
    Port: Twisted Pair
    PHYAD: 0
    Transceiver: internal
    Auto-negotiation: on
    MDI-X: off (auto)
    Supports Wake-on: umbg
    Wake-on: d
    Current message level: 0x00000007 (7)
                           drv probe link
    Link detected: yes
root@kali:~#
```

Figura 23 – Saída do comando ethtool eth0
Fonte: Elaborado pelo autor (2019)

```
root@kali:~# ethtool eth1
Settings for eth1:
    Supported ports: [ TP ]
    Supported link modes:   10baseT/Half 10baseT/Full
                           100baseT/Half 100baseT/Full
                           1000baseT/Full
    Supported pause frame use: No
    Supports auto-negotiation: Yes
    Supported FEC modes: Not reported
    Advertised link modes:  10baseT/Half 10baseT/Full
                           100baseT/Half 100baseT/Full
                           1000baseT/Full
    Advertised pause frame use: No
    Advertised auto-negotiation: Yes
    Advertised FEC modes: Not reported
    Speed: 1000Mb/s
    Duplex: Full
    Port: Twisted Pair
    PHYAD: 0
    Transceiver: internal
    Auto-negotiation: on
    MDI-X: off (auto)
    Supports Wake-on: umbg
    Wake-on: d
    Current message level: 0x00000007 (7)
                           drv probe link
    Link detected: yes
root@kali:~#
```

Figura 24 – Saída do comando ethtool eth1
Fonte: Elaborado pelo autor (2019)

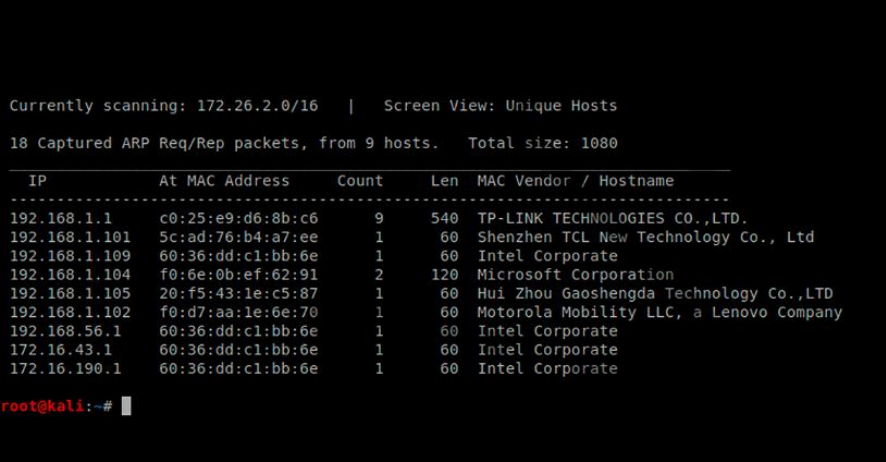
```
root@kali:~# ethtool lo
Settings for lo:
    Link detected: yes
root@kali:~#
```

Figura 25 – Saída do comando ethtool lo
Fonte: Elaborado pelo autor (2019)

Após obter os dados das interfaces de rede, iniciamos um escaneamento da rede com o comando *netdiscover -r*, com a finalidade de obter informações dos *hosts* ativos, conforme demonstrado na Listagem “Comando para escaneamento de hosts ativos na rede” e Figura “Saída do comando netdiscover”.

```
root@kali:~# netdiscover
# Comando utilizado para realizar um escaneamento dos hosts
ativos da rede.
```

Comando de prompt 13 – Comando para escaneamento de hosts ativos na rede
Fonte: Elaborado pelo autor (2019)



```
Currently scanning: 172.26.2.0/16 | Screen View: Unique Hosts
18 Captured ARP Req/Rep packets, from 9 hosts. Total size: 1080
```

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.1.1	c0:25:e9:d6:8b:c6	9	540	TP-LINK TECHNOLOGIES CO.,LTD.
192.168.1.101	5c:ad:76:b4:a7:ee	1	60	Shenzhen TCL New Technology Co., Ltd
192.168.1.109	60:36:dd:c1:bb:6e	1	60	Intel Corporate
192.168.1.104	f0:6e:0b:ef:62:91	2	120	Microsoft Corporation
192.168.1.105	20:f5:43:1e:c5:87	1	60	Hui Zhou Gaoshengda Technology Co.,LTD
192.168.1.102	f0:d7:aa:1e:6e:70	1	60	Motorola Mobility LLC, a Lenovo Company
192.168.56.1	60:36:dd:c1:bb:6e	1	60	Intel Corporate
172.16.43.1	60:36:dd:c1:bb:6e	1	60	Intel Corporate
172.16.190.1	60:36:dd:c1:bb:6e	1	60	Intel Corporate

```
root@kali:~#
```

Figura 26 – Saída do comando netdiscover
Fonte: Elaborado pelo autor (2019)

Observando a Figura “Saída do comando netdiscover”, verificamos a existência de diversos dispositivos que estão ativos na rede em questão.

A importância desse comando está em realizar uma verificação da presença de dispositivos estranhos ao ambiente em questão, que possivelmente poderiam estar realizando a coleta ou extração de informações sensíveis.

Como próximo passo, iniciaremos a verificação de informações sobre as conexões TCP/UDP estabelecidas pelo *host* ou rede objeto da investigação. Para isso, utilizaremos o comando *netstat -tupan* demonstrado na Listagem “Comando para verificar conexões estabelecidas” e Figura “Saída do comando netstat.”.

```
root@kali:~# netstat -tupan
# Comando utilizado para verificar as conexões TCP e UDP
estabelecidas pelo host.
```

Comando de prompt 14 – Comando para verificar conexões estabelecidas
Fonte: Elaborado pelo autor (2019)


```

root@kali:~# netstat -tupan
Conexões Internet Ativas (servidores e estabelecidas)
Proto Recv-Q Send-Q Endereço Local      Endereço Remoto      Estado      PID/Program name
tcp        0      0 192.168.1.103:33532  104.17.79.221:443    TIME_WAIT   -
tcp        0      0 192.168.1.103:41736  13.227.101.47:443    TIME_WAIT   -
tcp        0      0 192.168.1.103:42034  200.147.166.107:443  TIME_WAIT   -
tcp        0      0 192.168.1.103:35144  189.7.75.216:80      TIME_WAIT   -
tcp        0      0 192.168.1.103:50032  52.202.154.44:443    TIME_WAIT   -
tcp        0      0 192.168.1.103:44438  13.227.101.110:443   TIME_WAIT   -
tcp        0      0 192.168.1.103:50068  13.227.105.70:443    TIME_WAIT   -
tcp        0      0 192.168.1.103:55298  104.18.4.233:443     TIME_WAIT   -
tcp        0      0 192.168.1.103:47652  200.147.15.218:443   TIME_WAIT   -
tcp        0      0 192.168.1.103:46032  200.187.199.64:443   TIME_WAIT   -
tcp        0      0 192.168.1.103:60448  200.147.166.104:443  TIME_WAIT   -
tcp        0      0 192.168.1.103:40946  13.227.114.194:443   TIME_WAIT   -
tcp        0      0 192.168.1.103:44620  13.227.101.20:443    TIME_WAIT   -
tcp        0      0 192.168.1.103:60354  13.227.101.56:443    TIME_WAIT   -
tcp        0      0 192.168.1.103:43294  13.227.101.78:443    TIME_WAIT   -
tcp        0      0 192.168.1.103:37930  13.35.78.84:443      TIME_WAIT   -
udp        0      0 192.168.56.104:68    0.0.0.0:*            482/NetworkManager
udp        0      0 0.0.0.0:68          0.0.0.0:*            1516/dhclient
root@kali:~#

```

Figura 27 – Saída do comando netstat

Fonte: Elaborado pelo autor (2019)

De posse das conexões estabelecidas, precisamos coletar informações sobre a tabela de rotas do sistema, para analisarmos a existência de um redirecionamento de tráfego ou a de uma rota para uma rede desconhecida.

Para coletar essa informação, utilizamos o comando **route**, que retorna a tabela de rotas, conforme Listagem “Comando para verificar a tabela de rotas” e Figura “Saída do comando route”.

```

root@kali:~# route
# Comando utilizado para verificar a tabela de rotas do
sistema.

```

Comando de prompt 15 – Comando para verificar a tabela de rotas

Fonte: Elaborado pelo autor (2019)

```

root@kali:~# route
Tabela de Roteamento IP do Kernel
Destino      Roteador      MáscaraGen.      Opções Métrica Ref  Uso Iface
default      192.168.1.1    0.0.0.0          UG      0      0      0 eth0
192.168.1.0  0.0.0.0        255.255.255.0    U        0      0      0 eth0
192.168.56.0 0.0.0.0        255.255.255.0    U      100      0      0 eth1
root@kali:~#

```

Figura 28 – Saída do comando route

Fonte: Elaborado pelo autor (2019)

Para finalizar o processo de obtenção de dados da rede e *host* objeto da investigação, efetuamos o comando **traceroute** – direcionado ao site <http://www.uol.com.br> – ou a qualquer site da internet.

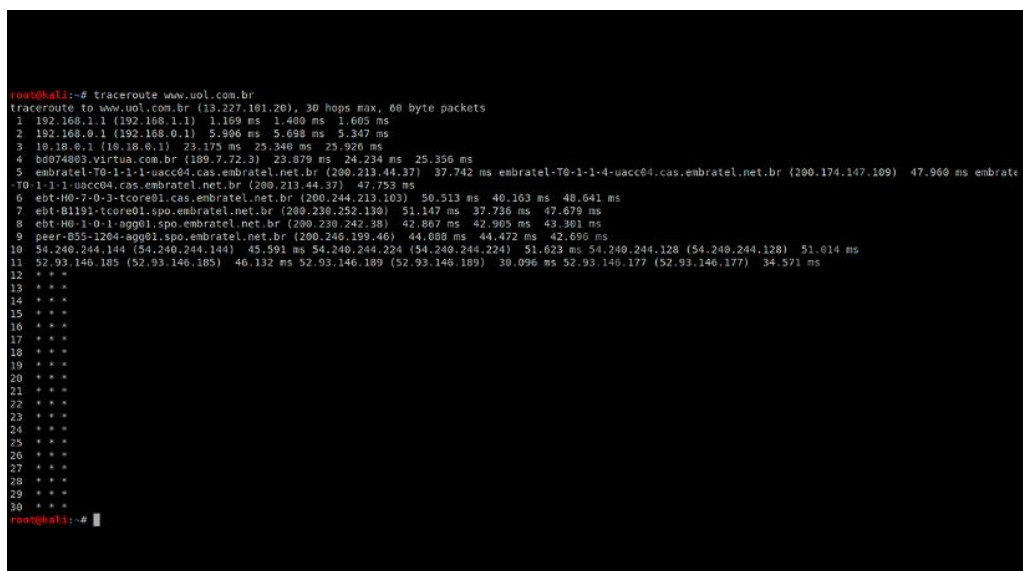
Seu objetivo principal é verificar a rota percorrida pelo pacote até seu endereço final, verificando a identificação de possíveis redirecionamentos de tráfegos, conforme

Listagem “Comando para verificar a rota de um pacote de rede” e Figura “Saída do comando traceroute”.

```
root@kali:~# traceroute www.uol.com.br
# Comando utilizado para verificar a rota do pacote de rede
até seu destino final.
```

Comando de prompt 16 – Comando para verificar a rota de um pacote de rede

Fonte: Elaborado pelo autor (2019)



```
root@kali:~# traceroute www.uol.com.br
traceroute to www.uol.com.br (13.227.161.20), 30 hops max, 60 byte packets
 1 192.168.1.1 (192.168.1.1)  1.169 ms  1.480 ms  1.605 ms
 2 192.168.0.1 (192.168.0.1)  5.906 ms  5.698 ms  5.347 ms
 3 10.18.0.1 (10.18.0.1)  23.175 ms  25.340 ms  25.920 ms
 4 bd074803.virtua.com.br (189.7.72.3)  23.879 ms  24.234 ms  25.356 ms
 5 embratel-T0-1-1-1-uacc04.cas.embratel.net.br (200.213.44.37)  37.742 ms embratel-T0-1-1-4-uacc04.cas.embratel.net.br (200.174.147.109)  47.960 ms embratel-T0-1-1-uacc04.cas.embratel.net.br (200.213.44.37)  47.753 ms
 6 ebt-H0-7-0-3-tcore01.cas.embratel.net.br (200.244.213.103)  50.513 ms  40.163 ms  48.641 ms
 7 ebt-B1191-tcore01.spo.embratel.net.br (200.230.252.130)  51.147 ms  37.736 ms  47.679 ms
 8 ebt-H0-1-0-1-agg01.spo.embratel.net.br (200.230.242.38)  42.867 ms  42.905 ms  43.301 ms
 9 peer-B05-1204-agg01.spo.embratel.net.br (200.246.199.46)  44.088 ms  44.472 ms  42.690 ms
10 54.240.244.144 (54.240.244.144)  45.591 ms  54.240.244.224 (54.240.244.224)  51.023 ms  54.240.244.128 (54.240.244.128)  51.014 ms
11 52.93.146.185 (52.93.146.185)  46.132 ms  52.93.146.189 (52.93.146.189)  30.096 ms  52.93.146.177 (52.93.146.177)  34.571 ms
12 * * *
13 * * *
14 * * *
15 * * *
16 * * *
17 * * *
18 * * *
19 * * *
20 * * *
21 * * *
22 * * *
23 * * *
24 * * *
25 * * *
26 * * *
27 * * *
28 * * *
29 * * *
30 * * *
```

Figura 29 – Saída do comando traceroute

Fonte: Elaborado pelo autor (2019)

As informações coletadas anteriormente possuem significativa importância em um processo investigativo de análise forense, pois, por meio delas, obtemos dados que nos direcionam na busca de artefatos de rede, identificando princípios e possíveis ações criminosas.

Cabe ressaltar que ações de cibercriminosos, *Insiders* e outros produzem muitos ruídos e deixam inúmeros rastros. O papel da Análise Forense de Tráfego de Rede é examinar e tentar obter o máximo de informações possíveis, cujo produto final é identificar o autor, vetor de um ataque.

5.5 Analisando tráfego com o Wireshark

Para a correta análise do tráfego de uma rede, é preciso que tenhamos o domínio dos protocolos básicos da pilha TCP/IP. Analisar tráfego significa observar os pacotes, procurando por algum comportamento fora do comum que corresponda a algum problema de rede ou até mesmo um ataque.

Para entender o quanto é importante a análise de tráfego, vamos observar o que está ocorrendo na captura **Analise_02.pcap**, para isso, vamos abrir o arquivo com o *Wireshark* e, já de início, constatamos um grande número de pacotes do protocolo *ARP* trafegando pela rede, conforme a Figura “Tela do Wireshark com pacotes da captura *Analise_02.pcap*”.

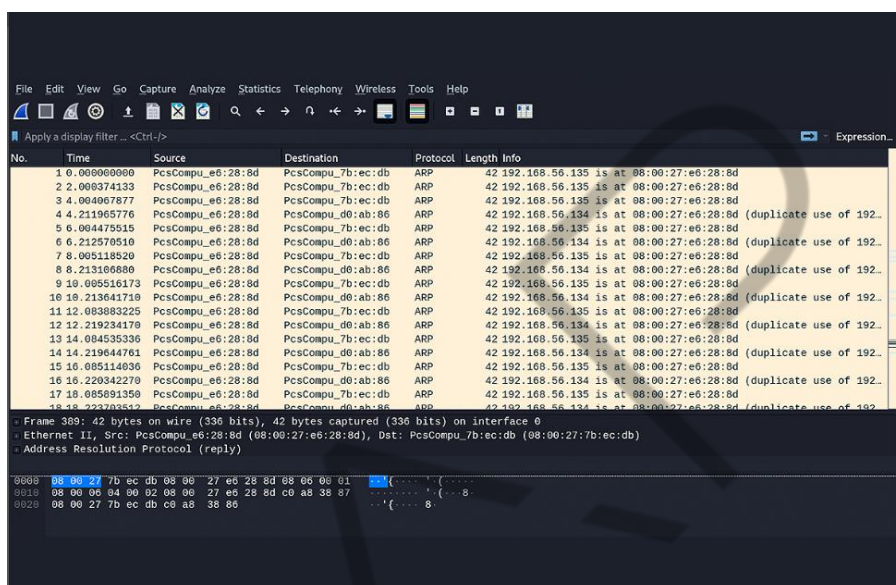


Figura 30 – Tela do Wireshark com pacotes da captura *Analise_02.pcap*
Fonte: Elaborado pelo autor (2019)

Analisando as primeiras três linhas da captura em questão: 1 0.000000000 PcsCompu_e6:28:8d PcsCompu_7b:ec:db ARP 42 192.168.56.135 is at 08:00:27:e6:28:8d.

Aqui, o host *PcsCompu_e6:28:8d* diz para o *PcsCompu_7b:ec:db* que seu IP é 192.168.56.135 e seu MAC é 08:00:27:e6:28:8d, já nas outras três linhas observamos uma repetição da anterior.

Na quarta linha, observamos o seguinte: 4 4.211965776 PcsCompu_e6:28:8d PcsCompu_d0:ab:86 ARP 42 192.168.56.134 is at 08:00:27:e6:28:8d (duplicate use of 192.168.56.135 detected!).

Nessa linha, temos o mesmo *host PcsCompu_e6:28:8d* dizendo para um *host* diferente (*PcsCompu_d0:ab:86*) que seu IP é 192.168.56.134, porém com o mesmo MAC 08:00:27:e6:28:8d.

Organizando tudo isso, teremos o Quadro “Organização dos dados da observação do tráfego do Wireshark”, apresentado abaixo.

Host	MAC	IP
Host A	PcsCompu_e6:28:8d	192.168.56.135
Host B	PcsCompu_7b:ec:db	Não é possível observar
Host C	PcsCompu_e6:28:8d	192.168.56.134
Host D	PcsCompu_d0:ab:86	Não é possível observar

Quadro 4 – Organização dos dados da observação do tráfego do Wireshark
 Fonte: Elaborado pelo autor (2019)

Até aqui, concluímos que o *Host A* e o *Host C* são iguais, pois ambos possuem o mesmo endereço *MAC*, porém, para o *Host B*, ele informa via protocolo *ARP* que seu IP é 192.168.56.135, enquanto para o *Host D*, informa que o endereço de rede é 192.168.56.134. Esse comportamento não é comum para o protocolo em questão. As próximas linhas são uma repetição do que foi observado nas linhas anteriores. Diante disso, o que essas informações querem nos mostrar?

Como já afirmado: *Host A = Host C*, assim é possível afirmar que até o presente momento temos *n* tráfego e *três Hosts*, sendo que um deles está enviando mensagens *ARP* com diferentes endereços IPs para a mesma máquina, esse comportamento produz, nos *hosts* que recebem essa informação, uma nova entrada na tabela *ARP*. Como resultado, o *host B* vai possuir em sua tabela a entrada apresentada na Figura “Entrada da tabela ARP do host B”.

```
? (192.168.56.135)_em 08:00:27:e6:28:8d [ether] em enp0s3
```

Figura 31 – Entrada da tabela ARP do host B
 Fonte: Elaborado pelo autor (2019)

Essa linha na tabela *ARP* do *Host B* informa que o host de MAC 08:00:27:e6:28:8d possui o IP 192.168.56.135, e sempre que precisar enviar algum pacote para esse endereço, ele utilizará o MAC relacionado.

Para o *Host D*, observamos o mesmo processo, o envio repetitivo da mensagem *ARP* fez com que o host alterasse sua tabela, ficando como mostra a Figura “Entrada da tabela ARP do host D”.

```
? (192.168.56.134)_em 08:00:27:e6:28:8d [ether] em enp0s3
```

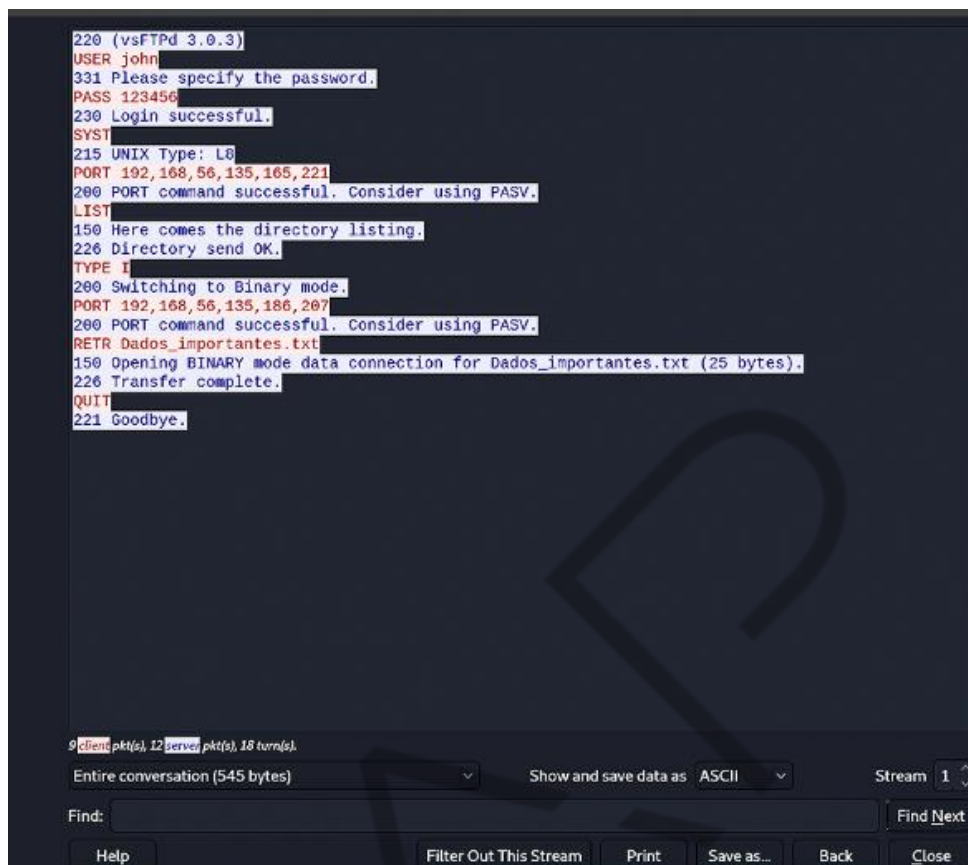
Figura 32 – Entrada da tabela ARP do host D
Fonte: Elaborado pelo autor (2019)

Com esses dados, pode-se afirmar que o host PcsCompu_e6:28:8d receberá todas as mensagens destinadas aos endereços IP 192.168.56.134 e 192.168.56.135, caracterizando um ataque de Arp Spoof, em que o atacante pôde capturar todo o tráfego entre esses dois *hosts* em questão.

Prosseguindo na análise das linhas do tráfego, percebemos que as mensagens do ataque ARP se repetem em aproximadamente 559 linhas, nas quais, em sequência, observamos a assinatura de um tráfego FTP, que pode ser o objetivo do atacante.

A ação maliciosa do host PcsCompu_e6:28:8d pode ter como intenção realizar a captura de credenciais do servidor FTP, uma vez que todos os pacotes transmitidos passaram pelo host malicioso antes de chegar ao destino, obtendo-se senhas e dados sensíveis.

No Wireshark, ao clicar com o botão direito sobre uma linha do fluxo FTP de dados e seguindo para a opção FOLLOW → TCP STREAM, podemos observar tudo o que foi transmitido entre o servidor FTP e a sua resposta, incluindo as credenciais (Figura “TCP STREAM Wireshark”).



```
220 (vsFTPd 3.0.3)
USER john
331 Please specify the password.
PASS 123456
230 Login successful.
SYST
215 UNIX Type: L0
PORT 192,168,56,135,165,221
200 PORT command successful. Consider using PASV.
LIST
150 Here comes the directory listing.
226 Directory send OK.
TYPE I
200 Switching to Binary mode.
PORT 192,168,56,135,186,207
200 PORT command successful. Consider using PASV.
RETR Dados_importantes.txt
150 Opening BINARY mode data connection for Dados_importantes.txt (25 bytes).
226 Transfer complete.
QUIT
221 Goodbye.
```

9 client pkt(s), 12 server pkt(s), 18 turn(s).

Entire conversation (545 bytes) Show and save data as ASCII Stream 1

Find: Find Next

Help Filter Out This Stream Print Save as... Back Close

Figura 33 – TCP STREAM Wireshark
Fonte: Elaborado pelo autor (2019)

Depreende-se que o tráfego descrito trata da assinatura de um ataque de *ARP Spoof*, e que devido a isso, o atacante em questão obteve acesso às credenciais e ao servidor *FTP*.

5.6 Utilizando as ferramentas Foremost e TCPFLOW para extração de arquivos

5.6.1 Instalação do Foremost e TCPFLOW

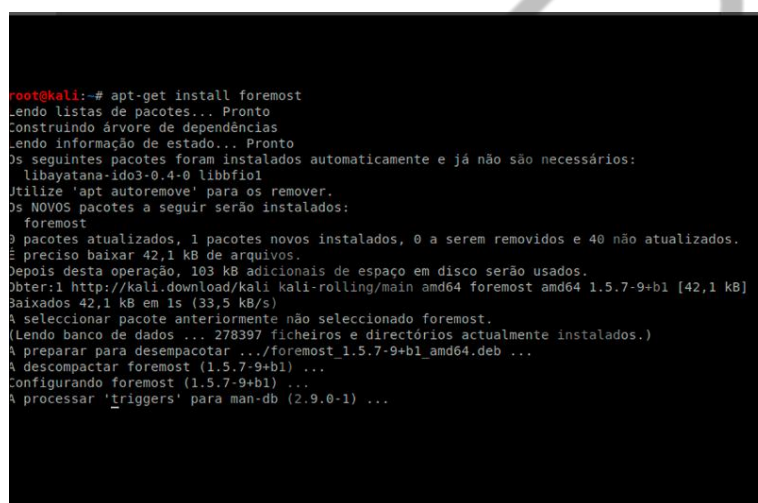
Foremost é um programa de console para recuperação de dados a partir de seus cabeçalhos, rodapés e estruturas de dados internas. A ação de busca e recuperação de arquivos também é conhecida como extração de dados (Data Carving) e trabalha com imagens geradas pelo comando *dd* e PCAPS de tráfegos capturados (ALMEIDA, 2018).

Segundo PELLIN (2002), o *Tcpflow* é um programa que captura dados transmitidos e os armazena para que possam ser facilmente analisados posteriormente, sendo possível reconstruir a sequência de dados de forma correta.

Para utilização dos programas em nosso laboratório, vamos realizar as instalações no *Kali Linux* fazendo uso dos comandos, conforme as Listagens e Figuras: “Instalação do Foremost”; “Saída do comando utilizado para instalação do Foremost” e “Instalação do TCPFlow”; “Saída do comando utilizado para realizar a instalação do TCPFlow”.

```
root@kali:~# apt-get install foremost
# Comando que realiza a instalação do Foremost via terminal do Linux
```

Comando de prompt 17 – Instalação do Foremost
Fonte: Elaborado pelo autor (2019)

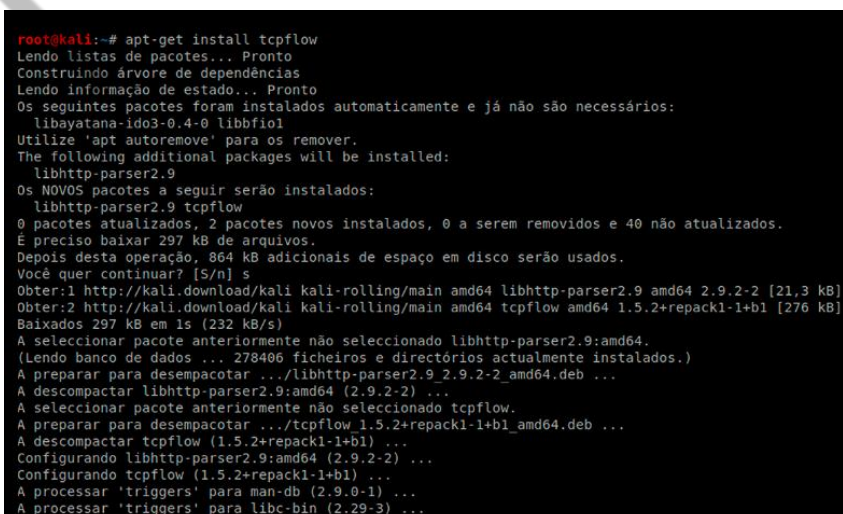


```
root@kali:~# apt-get install foremost
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Os seguintes pacotes foram instalados automaticamente e já não são necessários:
  libayatana-ido3-0.4-0 libbfiol
Utilize 'apt autoremove' para os remover.
Os NOVOS pacotes a seguir serão instalados:
  foremost
0 pacotes atualizados, 1 pacotes novos instalados, 0 a serem removidos e 40 não atualizados.
É preciso baixar 42,1 kB de arquivos.
Depois desta operação, 103 kB adicionais de espaço em disco serão usados.
Obter:1 http://kali.download/kali kali-rolling/main amd64 foremost amd64 1.5.7-9+b1 [42,1 kB]
Baixados 42,1 kB em 1s (33,5 kB/s)
A seleccionar pacote anteriormente não seleccionado foremost.
(Lendo banco de dados ... 278397 ficheiros e directórios actualmente instalados.)
A preparar para descompactar .../foremost_1.5.7-9+b1_amd64.deb ...
A descompactar foremost (1.5.7-9+b1) ...
Configurando foremost (1.5.7-9+b1) ...
A processar 'triggers' para man-db (2.9.0-1) ...
```

Figura 34 – Saída do comando utilizado para instalação do Foremost
Fonte: Elaborado pelo autor (2019)

```
root@kali:~# apt-get install tcpflow
# Comando utilizado para realizar a instalação da ferramenta TCPFlow
```

Comando de prompt 18 – Instalação do TCPFlow
Fonte: Elaborado pelo autor (2019)



```
root@kali:~# apt-get install tcpflow
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Os seguintes pacotes foram instalados automaticamente e já não são necessários:
  libayatana-ido3-0.4-0 libbfiol
Utilize 'apt autoremove' para os remover.
The following additional packages will be installed:
  libhttp-parser2.9
Os NOVOS pacotes a seguir serão instalados:
  libhttp-parser2.9 tcpflow
0 pacotes atualizados, 2 pacotes novos instalados, 0 a serem removidos e 40 não atualizados.
É preciso baixar 297 kB de arquivos.
Depois desta operação, 864 kB adicionais de espaço em disco serão usados.
Você quer continuar? [S/n] s
Obter:1 http://kali.download/kali kali-rolling/main amd64 libhttp-parser2.9 amd64 2.9.2-2 [21,3 kB]
Obter:2 http://kali.download/kali kali-rolling/main amd64 tcpflow amd64 1.5.2+repack1-1+b1 [276 kB]
Baixados 297 kB em 1s (232 kB/s)
A seleccionar pacote anteriormente não seleccionado libhttp-parser2.9:amd64.
(Lendo banco de dados ... 278406 ficheiros e directórios actualmente instalados.)
A preparar para descompactar .../libhttp-parser2.9_2.9.2-2_amd64.deb ...
A descompactar libhttp-parser2.9:amd64 (2.9.2-2) ...
A seleccionar pacote anteriormente não seleccionado tcpflow.
A preparar para descompactar .../tcpflow_1.5.2+repack1-1+b1_amd64.deb ...
A descompactar tcpflow (1.5.2+repack1-1+b1) ...
Configurando libhttp-parser2.9:amd64 (2.9.2-2) ...
Configurando tcpflow (1.5.2+repack1-1+b1) ...
A processar 'triggers' para man-db (2.9.0-1) ...
A processar 'triggers' para libc-bin (2.29-3) ...
```

Figura 35 – Saída do comando utilizado para realizar a instalação do TCPFlow

Fonte: Elaborado pelo autor (2019)

5.6.2 Extraíndo arquivos de uma captura de tráfego com o *Foremost* e *TCPFLOW*

Inicialmente, precisamos esclarecer o ambiente no qual foi obtida a captura, conforme demonstrado na Figura “Saída do comando utilizado para realizar a instalação do TCPFlow”.

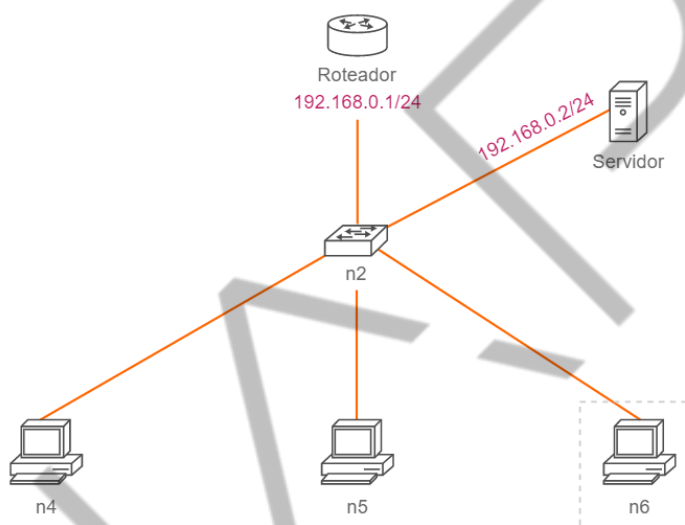


Figura 36 – Diagrama do ambiente de realização da captura cap_01.pcap

Fonte: Elaborado pelo autor (2019)

Para iniciar nossos trabalhos, tomemos a seguinte situação problema:

Conforme já tratado em tópicos anteriores e o conhecimento já adquirido do conceito de *Insiders*, um colaborador insatisfeito com as políticas de pessoal da empresa realizou, de forma não autorizada, um acesso ao servidor da empresa e o download de fotos e documentos.

Sendo assim, o responsável pela segurança da rede dessa empresa obteve uma captura de tráfego de um dos momentos em que aconteceu o fato.

Nesse contexto, vamos analisar o tráfego, determinar os hosts envolvidos e quais foram os dados obtidos pelo *Insider*, trabalhando no processo de extração e remontagem do pacote capturado, mais conhecido como *PCAP*.

Utilizaremos o arquivo **cap_01.pcap** obtido pela equipe de segurança de redes, do qual tentaremos extrair os arquivos que tenham sido transmitidos pela conexão estabelecida entre o *Insider* e o servidor de onde fez o download dos arquivos.

Após a realização da instalação das ferramentas *Foremost* e *TCPFlow*, daremos início ao processo de extração de dados capturados no arquivo **cap_01.pcap**, fornecido anteriormente.

Para execução da extração, utilizaremos o Quadro “Comandos para extração de arquivos do cap_01.pcap”, que contém a ordem e os comandos a serem executados durante o processo de *Carving*.

Comandos	Função dos Comandos
<code>mkdir tcpflow/</code>	Comando utilizado para criar uma pasta tcpflow.
<code>tcpflow -r cap_01.pcap -o tcpflow/</code>	Comando utilizado para executar a ferramenta tcpflow, que reordena o fluxo dos pacotes do pcap capturado. As opções -r leitura do arquivo e -o são utilizadas para direcionar a saída do programa para uma pasta de destino.
<code>mkdir arquivos_pcap</code>	Comando utilizado para criar uma pasta arquivos_pcap para os arquivos extraídos.
<code>cat tcpflow/* > arquivos_pcap/arquivos_pcap.dump</code>	Comando utilizado para ter, com o leitor de arquivos cat, todo o conteúdo existente na pasta tcpflow/, redirecionando a saída da leitura para um arquivo arquivos_pcap.dump, que estará localizado na pasta arquivos_pcap/ criada anteriormente.
<code>mkdir saida_arquivos_pcap/</code>	Comando utilizado para criar a pasta saida_arquivos_pcap.
<code>foremost -i arquivos_pcap/arquivos_pcap.dump -o saida_arquivos_pcap/</code>	Executando o comando do Foremost com a opção -i para extrair o conteúdo de arquivos_pcap.dump gerado anteriormente e redirecionar com a opção -o a saída dos arquivos obtidos para a pasta saida_arquivos_pcap/.
<code>rm -rf tcpflow/</code>	Comando utilizado para excluir a pasta tcpflow/ criada.
<code>rm -rf arquivos_pcap</code>	Comando utilizado para excluir a pasta arquivos_pcap gerada.

Quadro 5 – Comandos para extração de arquivos do cap_01.pcap

Fonte: Elaborado pelo autor (2019)

Tendo como ponto inicial a pasta **/root/Downloads/**, para a qual o arquivo foi baixado, iniciamos o processo de extração do conteúdo do **cap_01.pcap**, executando os comandos da Listagem “Extração de conteúdo com o Foremost e TCPFlow”.

```
root@kali:~/Downloads# mkdir tcpflow/
# Comando utilizado para criar uma pasta tcpflow
root@kali:~/Downloads# tcpflow -r cap_01.pcap -o tcpflow/
```



```
# Comando utilizado para executar a ferramenta tcpflow
ordenando o fluxo dos pacotes do pcap capturado e direcionando
saída do programa para uma pasta de destino.
reportfilename: tcpflow//report.xml
# Saída do comando anterior
root@kali:~/Downloads# mkdir arquivos_pcap
# Comando utilizado para criar uma pasta arquivos_pcap
root@kali:~/Downloads# cat tcpflow/* >
arquivos_pcap/arquivos_pcap.dump
# Comando utilizado para ler o conteúdo da pasta tcpflow e
direcionando a saída do comando para um arquivo .dump
root@kali:~/Downloads# mkdir saida_arquivos_pcap/
# Comando utilizado para criar uma pasta para receber os
arquivos extraídos
root@kali:~/Downloads# foremost -i
arquivos_pcap/arquivos_pcap.dump -o saida_arquivos_pcap/
# Comando utilizado para executar a ferramenta Foremost que
extraí o conteúdo do arquivo .dump e direciona a saída para a
pasta de arquivos.
Processing: arquivos_pcap/arquivos_pcap.dump
|*|
# Saída da execução do comando do Foremost
root@kali:~/Downloads# rm -rf tcpflow/
# Comando utilizado para remover a pasta tcpflow criada
anteriormente
root@kali:~/Downloads# rm -rf arquivos_pcap
# Comando utilizado para remover o arquivo .pcap criado
root@kali:~/Downloads#
```

Comando de prompt 19 – Extração de conteúdo com o Foremost e TCPFlow
Fonte: Elaborado pelo autor (2019)

Após a execução dos comandos da Listagem “Extração de conteúdo com o Foremost e TCPFlow”, o processo de extração dos arquivos encontra-se finalizado, bastando, agora, realizar um acesso na pasta: **/root/Downloads/saida_arquivos_pcap/**, bem como a análise do conteúdo extraído, conforme as Figuras “Arquivos extraídos no pcap”, “Arquivos .jpg extraídos no processo”, “Arquivos .pdf extraídos no processo”, “Arquivo de planta baixa capturado”, “Arquivos .png recuperados do pcap” e “Imagem de projeto residencial obtido na extração”.

Ao realizar o acesso ao diretório, observamos que o conteúdo extraído está organizado em pastas, que estão nomeadas por extensão encontrada, conforme a Figura “Arquivos extraídos no pcap”.

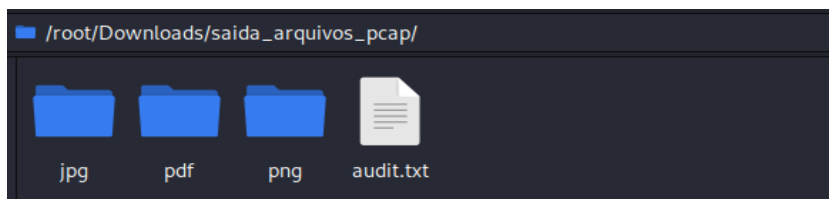


Figura 37 – Arquivos extraídos no pcap
Fonte: Elaborado pelo autor (2019)

Acessando a pasta **jpg/**, encontramos dois arquivos – 00001525.jpg e 00001647 – que aparentam ser duas imagens .jpg, conforme a Figura “Arquivos .jpg extraídos no processo”.

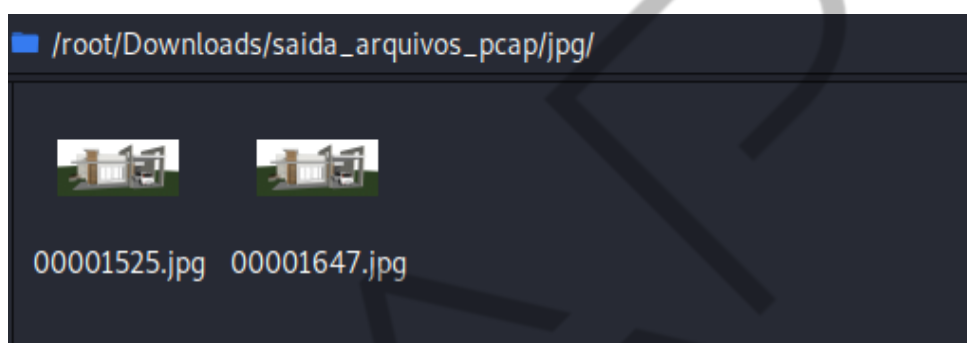


Figura 38 – Arquivos .jpg extraídos no processo
Fonte: Elaborado pelo autor (2019)

Partindo para o acesso à pasta **pdf/**, observamos a existência de dois arquivos .pdf – 00000000.pdf e 00001619.pdf –, conforme a Figura “Arquivos .pdf extraídos no processo”.

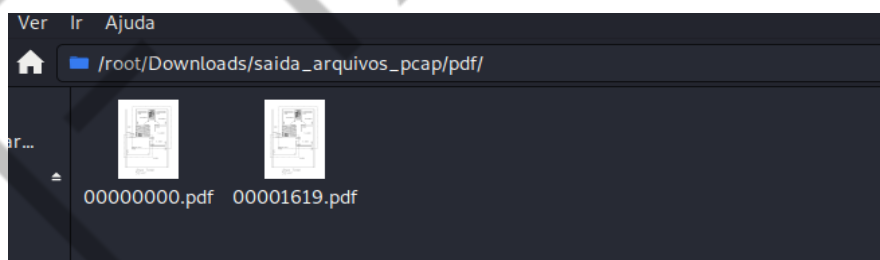


Figura 39 – Arquivos .pdf extraídos no processo
Fonte: Elaborado pelo autor (2019)

Abrindo o arquivo 00000000.pdf, percebemos que, aparentemente, esse conteúdo trata de uma planta baixa, conforme a Figura “Arquivo de planta baixa capturado”.

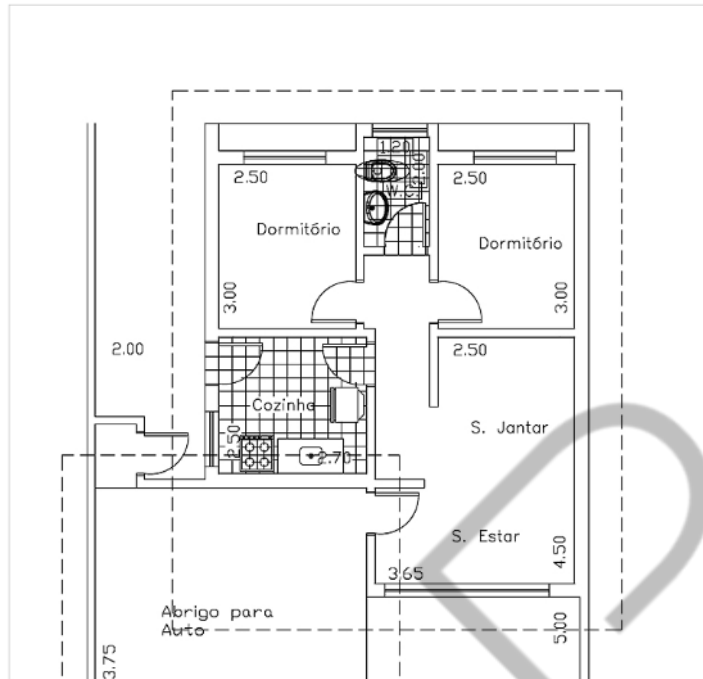


Figura 40 – Arquivo de planta baixa capturado
Fonte: Elaborado pelo autor (2019)

Em uma última verificação, acessamos a pasta **png/** e observamos a existência de dois arquivos de imagem, conforme a Figura “Arquivos .png recuperados do pcap”.

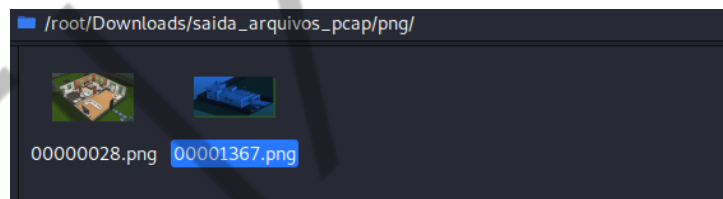


Figura 41 – Arquivos .png recuperados do pcap
Fonte: Elaborado pelo autor (2019)

Realizando o acesso à imagem 00001367.png, vemos o projeto de uma construção, conforme Figura “Imagem de projeto residencial obtido na extração”.



Figura 42 – Imagem de projeto residencial obtido na extração
Fonte: Elaborado pelo autor (2019)

Após a finalização do processo, percebemos a eficácia da utilização das ferramentas *Foremost* e *TCPFlow* durante a execução de uma Análise Forense em uma captura de tráfego de rede, mostrando que a comunicação entre nós de uma rede possui informações fundamentais em um processo de perícia.

5.7 Utilização da ferramenta *NetworkMiner*

5.7.1 Instalação

Para utilização do programa em nosso laboratório, necessitamos da instalação de um pacote chamado *Mono*, pré-requisito para instalação e funcionamento da ferramenta, para tanto, faremos uso dos comandos apresentados na Listagem “Instalação do Mono e NetworkMiner”.

```
root@kali:~# sudo apt-get install libmono-system-windows-
forms4.0-cil
# Comando utilizado para instalação de pacotes do Mono
root@kali:~# sudo apt-get install libmono-system-web4.0-cil
# Comando utilizado para instalação de pacotes do Mono
root@kali:~# sudo apt-get install libmono-system-net4.0-cil
# Comando utilizado para instalação de pacotes do Mono
root@kali:~# sudo apt-get install libmono-system-runtime-
serialization4.0-cil
# Comando utilizado para instalação de pacotes do Mono
root@kali:~# sudo apt-get install libmono-system-xml-linq4.0-
cil
# Comando utilizado para instalação de pacotes do Mono
root@kali:~# mono -version
# Comando utilizado para verificar a versão e instalação do
Mono
root@kali:~# wget
https://www.netresec.com/?download=NetworkMiner -O /tmp/nm.zip
# Comando utilizado para efetuar o download da ferramenta
NetworkMiner
root@kali:~# sudo unzip /tmp/nm.zip -d /opt/
```

Comando de prompt 20 – Instalação do Mono e NetworkMiner

Fonte: Elaborado pelo autor (2019)

5.7.2 Extraindo arquivos de um *pcap* com a ferramenta *networkminer*

O ambiente no qual foi obtida a captura está apresentado na Figura “Diagrama do ambiente de realização da captura cap_01.pcap”.

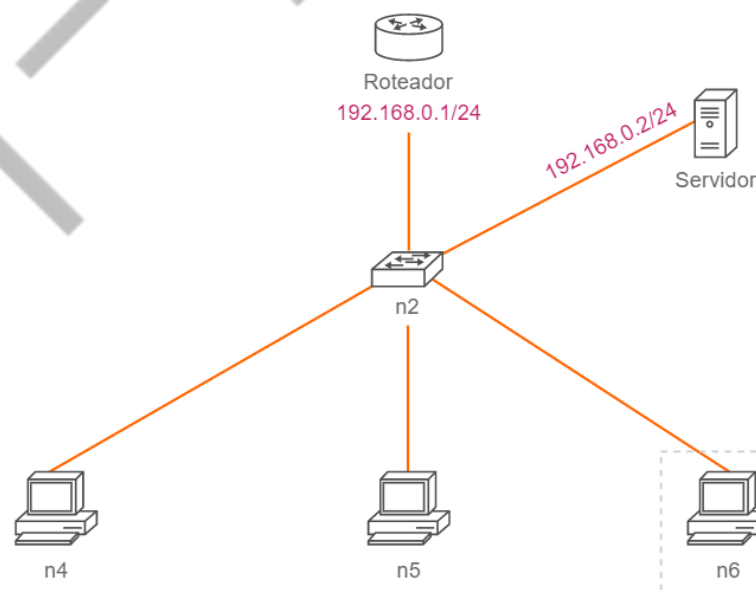


Figura 43 – Diagrama do ambiente de realização da captura cap_01.pcap

Fonte: Elaborado pelo autor (2019)

Para iniciar nossos trabalhos, tomemos a seguinte situação problema:

Conforme já tratado em tópicos anteriores e o conhecimento já adquirido do conceito de *Insiders*, um colaborador insatisfeito com as políticas de pessoal da empresa realizou, de forma não autorizada, um acesso ao servidor da empresa, fazendo o download de fotos e documentos.

Sendo assim, o responsável pela segurança da rede dessa empresa obteve uma captura de tráfego de um dos momentos em que aconteceu o fato.

Nesse contexto, vamos analisar o tráfego, determinar os hosts envolvidos e quais foram os dados obtidos pelo *Insider*, trabalhando no processo de extração e remontagem do pacote capturado, mais conhecido como *PCAP*.

Utilizaremos o arquivo **cap_01.pcap** obtido pela equipe de segurança de redes, do qual tentaremos extrair os arquivos que tenham sido transmitidos pela conexão estabelecida entre o *Insider* e o servidor de onde fez o download dos arquivos.

Contudo, cabe ressaltar que desconhecemos quais tipos de arquivos foram acessados ou obtidos e se são de texto, áudio ou imagem.

Para este exercício vamos utilizar a ferramenta simples e que facilita o trabalho denominada *Networkminer*, utilizada para remontagem e extração de dados de pacotes de rede.

Fazendo uso de um terminal do Sistema Operacional *Kali Linux* de nosso laboratório, executamos um comando para acessar a pasta onde se encontra o arquivo fornecido, conforme a Listagem “Comando utilizado acessar o diretório do arquivo cap_01.pcap”.

```
root@kali:~# /opt/networkminer/networkminer cap_01.pcap
```

Comando de prompt 21 – Comando utilizado acessar o diretório do arquivo cap_01.pcap
Fonte: Elaborado pelo autor (2019)

O resultado que se observa na Figura “Imagem da ferramenta *NetworkMiner*” é que cada aba do programa está repleta de informações que vão desde os endereços de rede dos hosts envolvidos, até o os arquivos que foram baixados pelo alvo no servidor.

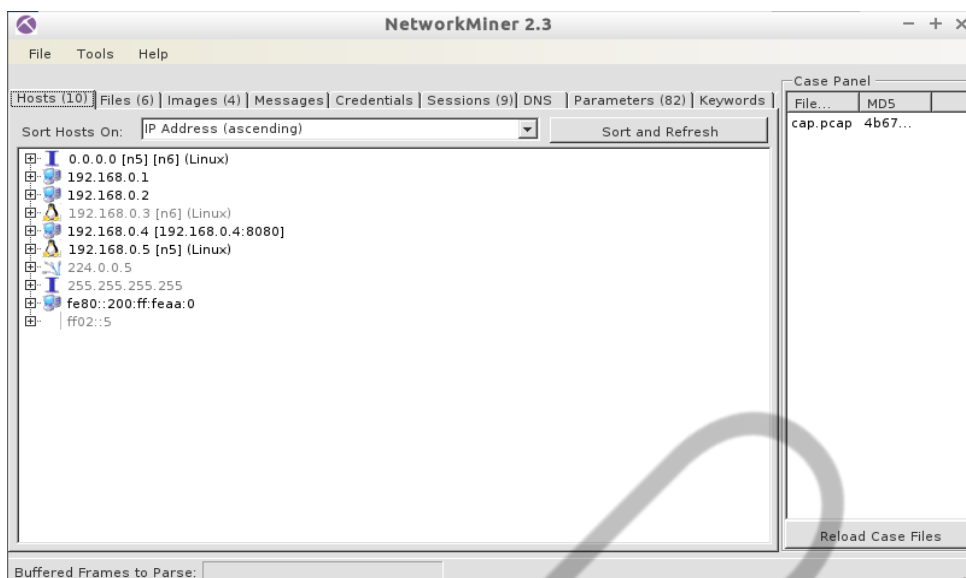


Figura 44 – Imagem da ferramenta NetworkMiner
Fonte: Elaborado pelo autor (2019)

Observando a Figura “Imagem da ferramenta NetworkMiner”, vemos duas abas que nesta análise são importantes: **files** e **imagens**. Na primeira, estarão todos os arquivos que o programa conseguiu realizar a extração; e na segunda, estarão as imagens. Acessando a primeira, podemos ver arquivos do tipo **pdf**, **png** e **jpeg**.

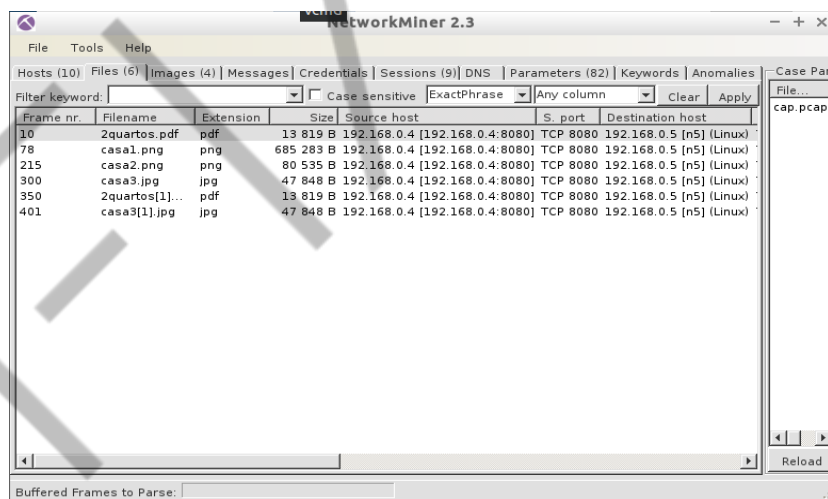


Figura 45 – Imagem da ferramenta NetworkMiner
Fonte: Elaborado pelo autor (2019)

Analisando o arquivo **2quartos.pdf**, podemos observar na Figura “Arquivo 2quartos.pdf” qual o arquivo que foi baixado, dessa forma, pela análise forense, podemos afirmar que o PC com endereço **192.168.0.5** fez o download de um arquivo do servidor com endereço **192.168.0.4**.

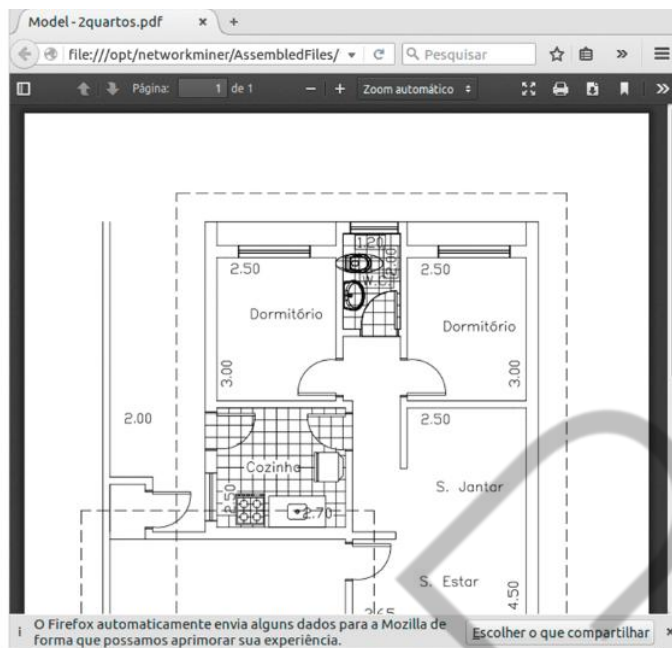


Figura 46 – Arquivo 2quartos.pdf
Fonte: Elaborado pelo autor (2019)

Na Figura “Arquivo 2quartos.pdf”, observamos a aba images, e nela encontramos apenas os arquivos de imagens, nos quais é oferecida uma pré-visualização, em miniaturas, das imagens que foram adquiridas pelo *Insider*.

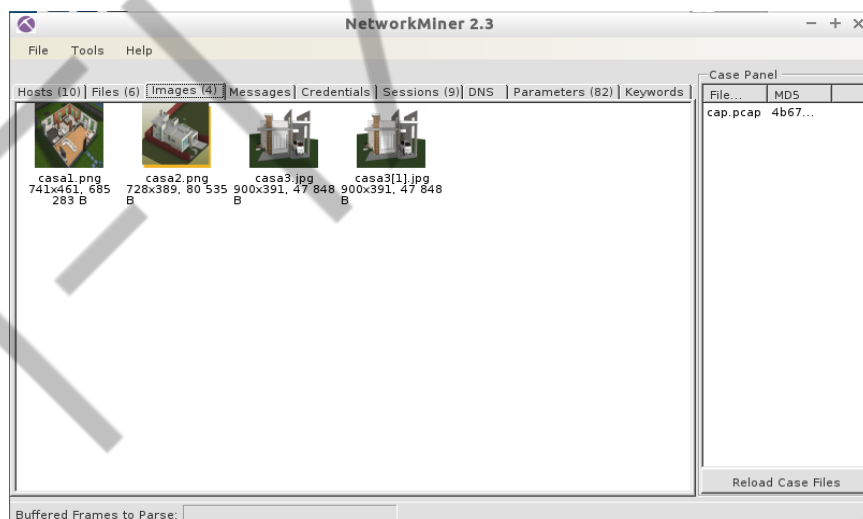


Figura 47 – Arquivo 2quartos.pdf
Fonte: Elaborado pelo autor (2019)

Considerando que o usuário *Insider* não tinha permissão para acessar os dados, constatamos que realmente houve um *download* de arquivos de imagens e *pdfs* do servidor (192.168.0.5) para o host (192.168.0.4).

Sendo assim, a Análise Forense de uma captura de tráfego de rede, diante de um processo judicial, pode ser peça fundamental, se usada como prova do ocorrido, para corroborar as suspeitas iniciais.

EMANIP

REFERÊNCIAS

ALMEIDA, R. Q. **Foremost e Scalpel**. Recuperação de arquivos. 2018. Disponível em: <<http://www.linhadecodigo.com.br/artigo/2968/foremost-e-scalpel-recuperacao-de-arquivos.aspx>>. Acesso em: 15 dez. 2019.

ARRUDA, O. D. R. **Preservação de evidências digitais em sistemas *in vivo***. Uma proposta para automatização do processo de preservação em sistemas Linux. Ilhas Maurício: Novas Edições Acadêmicas, 2018.

CASTRO JÚNIOR, A. P.; LISITA, B. L.; MOURA, T. S. M.; PINTO, T. J. **Forense Computacional em Memória Principal**. 2012. Disponível em: <<https://pt.scribd.com/document/517417003/Forense-Computacional-Em-Memoria-Principal>>. Acesso em: 28 maio 2021.

EDRM. **EDRM Model**. Disponível em: <<https://www.edrm.net/frameworks-and-standards/edrm-model/>>. Acesso em: 28 maio 2021.

FARMER, D.; VENEMA, W. **Perícia Forense Computacional – Teoria e Prática Aplicada**. 4. Reimpressão. São Paulo: Pearson Prentice Hall, 2011.

HALE-LIGH, M.; CASE, A.; LEVY, J.; WALTERS, A. **The Art of Memory Forensics**. Detecting Malware and Threats in Windows, Linux, and Mac Memory. Indianapolis: John Wiley & Sons Inc., 2014.

HAUSKNECHT, K.; FOIT, D.; BURIC, J. RAM data significance in digital forensics. **38th International Convention on Information and Communication Technology, Electronics and Microelectronics**, MIPRO 2015. Opatija, Croatia, May 2015, p. 1372-1375.

HORSWELL, J. **The Practice of Crime Scene Investigation**. New York: CRC Press, 2004.

JONES, J.; ETZKORN, L. Analysis of Digital Forensics Live System Acquisition Methods to Achieve Optimal Evidence Preservation. **SoutheastCon**, p. 1-6, 2016.

LAZARIDIS, I.; ARAMPATZIS, T. POURROS, S. **Evaluation of Digital Forensics Tools on Data Recovery and Analysis**. Proceedings of the Third International Conference on Weblogs and Social Media, Thessaloniki, p. 67-71, may, 2009.

NUNES, C. M. **A Análise de Tráfego em Redes como Método de Coleta de Evidências Digitais**: Estudo de Casos com Softwares de Análise de Tráfego. 2017. Disponível em: < <https://repositorio.animaeducacao.com.br/items/7f9d63b2-4d8f-4dc7-8018-383472e8f355/full> >. Acesso em: 15 dez. 2019.

OLIVEIRA, R. S.; CARISSIMI, A. S.; TOSCANI, S. S. **Sistemas Operacionais**. 4. ed. Porto Alegre, RS: Bookman: Instituto de Informática de UFRGS, 2010.

PELLIN, A. F. **Um Monitor de Transações de Serviços Internet**. Dissertação (Mestrado em Ciência da Computação), Universidade Federal do Rio Grande do Sul, Porto Alegre, 2002.

VANDEVEN, S. **Forensic Images: For Your Viewing Pleasure**. 2014. Disponível em: <<https://www.sans.org/white-papers/35447/>>. Acesso em: 28 maio 2021.

WEIDMAN, G. **Testes de Invasão**. Uma introdução prática ao hacking. São Paulo: Novatec, 2016.

EMANIP